Belief in Lax Logic

Matthew Fairtlough
matthew@fairflow.co.uk
Independent researcher
United Kingdom

Abstract

We propose a doxastic reading of Propositional Lax Logic (PLL), the intuitionistic modal logic of Fairtlough and Mendler whose single modality $\bigcirc$ has aspects both of possibility and of necessity: we read $\bigcirc M$ as "M is believed", by a single idealised believer whose belief is disciplined by evidence. The reading is defended by a chain of machine-checked results. On the negative side, over classical logic the reading is empty: on a Boolean algebra every nucleus is closed, so a classical belief modality collapses to $\bigcirc M \equiv M \vee a$ for a fixed proposition a. Over intuitionistic logic belief is rich, and its richness is evidential in a precise sense inherited from the constraint reading.

The paper's technical core is a new realisability semantics for PLL over the constraint models of the original paper, in which atoms carry evidence from a partial applicative structure and the clauses for $\twoheadrightarrow$ and $\bigcirc$ differ in how much a realiser is given about the future of the inquiry. Three clause families arise — uniform, strategy, and presented-strategy — and a sequence of separation theorems shows the choice determines the logic: rigid evidence validates schemes PLL does not prove; strategy evidence restores the countermodels but provably cannot support a completeness theorem, by an obstruction inherent in strategy evidence; presenting the future at implications yields completeness. The countermodels are supplied by a finitisation of the original canonical-model construction in which Zorn's lemma is replaced by an iterated appeal to PLL's own mechanised decision procedure, and the resulting models are executable: a verified checker certifies them by computation, and each world reads as a belief state — what is believed, and what is promised never to be evidenced.

Every result is formalised in Lean 4; the axiom footprint of each theorem is tracked, and the greater part of the development, including the entire decoration argument, is choice-free. We conclude with an account of the method, in which several of the paper's results were found by the failure, itself provable, of the theorems we first attempted.

CCS Concepts: • **Theory of computation** → **Modal and temporal logics**; **Constructive mathematics**; *Logic and verification.*

Keywords: lax logic, doxastic logic, nuclei, realisability, canonical models, decidability, Craig interpolation, Lean, machine-checked proof

ACM Reference Format:
Matthew Fairtlough. 2027. Belief in Lax Logic. In *Proceedings of Certified Programs and Proofs (CPP '27).* ACM, New York, NY, USA, 20 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

The object of this paper is a reading of the modality $\bigcirc$ characterised by the axiom schemes

```
○R : M ↠ ○M
○M : ○○M ↠ ○M
○S : (○M ∧ ○N) ↠ ○(M ∧ N)
```

with Modus Ponens and the rule "from $M \twoheadrightarrow N$ infer $\bigcirc M \twoheadrightarrow \bigcirc N$". The introduction of [24] motivated $\bigcirc$ by a list of six perspectives — Curry's 1948 lectures [14, 15], nuclei on complete Heyting algebras [39, 48, 55], Kripke and J-space semantics [32, 43, 59], the strong monads of computational type theory [11, 42, 51, 54], the proof-theoretic constraint reading [26], and the timing analysis of combinational circuits [50] — and settled on the constraint reading: $\bigcirc M$ as "for some constraint c, formula M holds under c". This paper adds a seventh reading and pursues it:

(7) $\bigcirc M$ is "**M is believed**". The unit $\bigcirc R$ says "what is true is believed"; the multiplication $\bigcirc M$ says "what is believed to be believed is believed"; $\bigcirc S$ says "beliefs combine". The reading is doxastic and not epistemic: nothing gives $\bigcirc M \twoheadrightarrow$ M, and indeed $\bigcirc\bot$ is consistent — a believer may hold an absurd belief without thereby believing everything. An inconsistent belief does not entail every formula.

Two features distinguish the present pursuit of (7) from a routine exercise in modal-logic reinterpretation, and they are connected.

The first is that the reading is justified by *theorems*, the central ones new. Classically the reading is degenerate: every

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
CPP '27, TBD
© 2027 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

nucleus on a Boolean algebra is closed, so a classical $\bigcirc$ has the single form $\bigcirc M \equiv M \vee \bigcirc\bot$, fixed by the one proposition $\bigcirc\bot$, its "credulity level". The total sceptic ($\bigcirc\bot \equiv \bot$, so $\bigcirc M \equiv M$) and the totally credulous believer ($\bigcirc\bot \equiv \top$, so $\bigcirc M \equiv \top$) are its extremes; every classical believer has this same form, differing only in the disjoined proposition, so there is no structural variety among them. A genuine spectrum of distinct believers appears only over intuitionistic logic — open nuclei, invisible classically, model belief under a hypothesis, and the closed $\bigcirc$-fragment of even one free algebra is infinite — and, by the context-completeness theorem descending from Curry's problem, a proof of $\bigcirc M$ contains the evidence for believing M. Belief backed by such evidence is *evidential* belief, and evidential belief is non-degenerate only over intuitionistic logic. Making this argument precise occupies Sections 3–6, whose main novelty is a realisability semantics in which this claim becomes a completeness theorem: PLL is exactly the logic of belief whose evidence may react to the future of the inquiry.

The second feature is methodological. Every claim in this paper is machine-checked, in a Lean 4 formalisation that also contains the whole of [24] — the calculi and their equivalences, cut elimination, the constraint semantics, soundness, completeness, decidability — and the axiomatic footprint of every theorem is tracked, most of the new material being choice-free. We do not regard the formalisation as an audit performed after the mathematics. Three times in this development the attempt to prove a plausible statement produced instead a proof of its negation, and each failure redirected the theory; the most consequential of these, the obstruction of Section 5, is the reason the completeness-grade semantics has the shape it has. In our experience the proof assistant reliably exposes cases we had overlooked. Section 9 reports the method, including the division of labour between the human author and the language model that operated the proof assistant.

The paper is organised as follows. Section 2 fixes the mechanised base and recalls the constraint semantics. Section 3 proves the classical collapse and the constructive richness. Section 4 defines evidence and the three realisability clause families; Section 5 proves the separation theorems and the obstruction; Section 6 the completeness theorem, in three steps — decoration, the finitised canonical model, and their composition. Section 7 runs the theory: executable countermodels read as belief states, and a proof compiled to its evidence. Sections 8–10 treat related work, method, and open problems.

2 The mechanised base

The formulas of PLL are generated by the grammar

```
M ::= A | ⊥ | M ∧ M | M ∨ M | M ↠ M | ○M
```

over a countable supply of propositional constants; ¬M abbreviates M ↠ ⊥.

Notation. Implication is written $M \twoheadrightarrow N$, where [24] writes $M \supset N$, and falsity is written $\bot$, where that paper writes *false*. The double-headed arrow is the symbol of the Lean artefact: the plain arrow is Lean's function type, and in the listings it has that meaning only. The artefact's formulas are the terms of an inductive type `PLLFormula` [D1], with the notation below defined over its constructors; the listings of Appendix A write the constructors in full. File names are given relative to the directory `LaxLogic/` of the artefact.

this paper	Lean notation	constructor of `PLLFormula`
A (a constant)		`.prop a`
$\bot$	$\bot$	`.falsePLL`
$M \wedge N$	$M \wedge N$	`.and M N`
$M \vee N$	$M \vee N$	`.or M N`
$M \twoheadrightarrow N$	$M \twoheadrightarrow N$	`.ifThen M N`
$\bigcirc M$	$\bigcirc M$	`.somehow M`
$\Gamma \vdash M$	$\Gamma \vdash M$	`LaxND Γ M`

The artefact presents PLL as a natural deduction system with membership-based contexts (`LaxND` [D2]), as an intrinsically-typed term calculus in the sense of the computational lambda-calculus (`Tm` [D3]), as a Hilbert system, and as a cut-free single-succedent sequent calculus (`SC` [D4]); all four are proved equivalent; cut elimination, the disjunction property, $\bigcirc$-reflection and conservativity over intuitionistic propositional logic ([D5]) are theorems, and the term calculus is strongly normalising for β together with the monadic associativity law, by a $\top\top$-lifting argument [46] that appears to be the first mechanised one with sums (`PLL/Normalisation/TopTop.lean`). Decidability is likewise mechanised, as a terminating, kernel-honest procedure over a contraction-free calculus [18, 38] — with the wrinkle, documented in the artefact and communicated separately to the calculus's author, that the published G4-style calculus for PLL is provably incomplete and a small repair is both necessary and sufficient. Craig interpolation holds, by a Maehara induction over the cut-free calculus, stated choice-free through an explicitly constructive finite-set toolkit ([D6], Appendix A); uniform interpolation, by contrast, remains open (Section 10). After an axiom-hygiene pass, the entire proof-theoretic chain — interpolation included — audits at `[propext, Quot.sound]`: no choice.

The semantics is that of [24], Definitions 3.1–3.2, mechanised verbatim (`PLL/Semantics/Kripke.lean`):

```
structure ConstraintModel where
  W : Type
  Ri : W → W → Prop   -- information order
  Rm : W → W → Prop   -- constraint order; Rm ⊆ Ri
  F : Set W           -- fallible worlds
  V : String → Set W  -- valuation; full on F
  ...                 -- eight structural conditions
```

Forcing is as in intuitionistic logic on the frame (W, R_i, V), with $\bot$ holding exactly at the fallible worlds, and the modal clause is the $\forall\exists$ statement

```
w ⊨ ○N  iff  for all v with w Rᵢ v
             there exists u with v Rₘ u, u ⊨ N
```

The clause endows ○ with properties both of possibility and of necessity, as the original paper observed. Under reading (7) it is a stability statement. Call N *credible* at a world v when a constraint step from v reaches a world satisfying N:

```
N is credible at v  iff  ∃ u. v Rₘ u and u ⊨ N
```

Then w ⊨ ○*N* says exactly that N *remains credible in all future worlds*: however inquiry develops, some admissible discharge of the constraint still verifies N. The fallible worlds — worlds with inconsistent information, validating everything — are what keep ○⊥ satisfiable, and under (7) they are the semantic counterpart of an inconsistently held belief. Soundness for sequents is [D7] (`PLL/Semantics/Kripke.lean`, audit `[propext]`); the classical completeness theorem of [24, §4] is [D8] (`PLL/Semantics/Completeness.lean`), kept in the artefact deliberately, Zorn's lemma included, as the foil for Section 6.2.

3 Classical belief collapses; constructive belief is rich

We first make precise the claim that reading (7) has no classical content.

Theorem 3.1 ([D9], `nucleusOrderIsoBot` [D10]). *On a Boolean algebra every nucleus is closed: $j\,x = x \sqcup j\bot$. The nuclei on B form a lattice order-isomorphic to B itself.*

Under reading (7), a classical believer is determined by one proposition, a = ○⊥, the strongest thing believed for no reason; belief is then "M, or a", which adds a to every proposition alike. At a = ⊥ the believer is the total sceptic, believing only what is true; at a = true, the totally credulous believer. Nothing between these two distinguishes one classical believer from another except the choice of a. Classically there is no structure of belief, only a degree of laxity. (The algebraic fact is classical [10, 48], with the modern treatment of [7]; the mechanisation, the sharp order-isomorphism, and the reading are what is added here.)

Theorem 3.2 ([D11], [D12]). *Intuitionistically, the open nucleus $u_a = (a \twoheadrightarrow \cdot)$ coincides with a closed nucleus exactly when excluded middle holds at a; and the closed ○-fragment RN(○, {}) of the free algebra on no generators is infinite.*

The open nucleus is belief under the hypothesis a — the logic of conditional commitment, "believing M modulo a" — and classically it is indistinguishable from a closed nucleus, precisely because $a \twoheadrightarrow M$ and M ∨¬a coincide there. That the closed fragment of even the generator-free algebra is infinite says the constructive believer has infinitely many pairwise inequivalent modal degrees where the classical believer had a single parameter (the embedding is through the Rieger–Nishimura lattice [53, 57]; nuclei over such non-complete Heyting algebras are studied in [19, 20]).

The smallest algebra already exhibits the separation, and the artefact enumerates it exactly (`Belief/Examples.lean`, kernel-checked by `decide`): on the three-element chain there are precisely four believers ([D13]) — the sceptic, the credulous believer, one dogmatic (closed) and one hypothetical (open) — and the last two differ ([D14]). Hypothetical belief is a new attitude from the first non-Boolean algebra onwards.

Theorem 3.3 (context completeness). *PLL ⊢ φ iff for every standard constraint context C, IPL ⊢ φ^C; and no finite set of constraint contexts suffices. ([D15], [D16].)*

This — mechanised after the Curry-problem analysis of TYPES 2000 [25] — is the theorem that makes "evidential" more than a slogan: a proof of ○*M* is, uniformly in the constraint, a proof of M under that constraint — the grounds are contained in the belief. Taken together, Theorems 3.1–3.3 establish the paper's central argument: classically belief collapses; constructively it is rich; the richness is evidential; and so a non-degenerate logic of evidential belief is possible only over intuitionistic logic. One more deserves recording here: the standard constraint contexts themselves form a Boolean algebra ([D17]) — the space of a believer's possible grounds is classical even though the belief those grounds discipline is not.

Proposition 3.4 (the doxastic profile). *○ is a normal modality (it validates K, and ○⊤ = ⊤); the derived rule "from Γ ⊢ M infer ○Γ ⊢ ○M" holds — logical omniscience; introspection is exact, ○○M ⊣⊢ ○M; and there is no factivity and no D axiom: ⊬ ¬ ○ ⊥ and ⊬ ○⊥, while ○⊥ ⊢ ○M for every M. (`Belief/Idealisation.lean`, `Belief/Normality.lean`, `Belief/Falsum.lean`.)*

This, following Hintikka [34], is the idealisation adopted openly: the unit says the believer accepts what is true, and omniscience over consequence is the price of being a closure operator. The underivabilities are by the countermodels of [24, Fig. 3]; the last clause is the credulous collapse, confined to the ○-level — a believer of the absurd believes everything, and nothing is thereby true.

4 Evidence

We now equip the constraint models with evidence; the ancestry is Kleene realisability [41, 62–64] in its Kripke-indexed form [47] (evidence-function possible-world models [28, 49] are closely related), with partial applicative structures the standard carriers [69]. The equipment is deliberately light.

```
structure Pca where
  Carrier : Type
  app : Carrier → Carrier → Option Carrier
  pair : Carrier → Carrier → Carrier
  fst snd : Carrier → Carrier
  tag : Bool → Carrier → Carrier
  untag : Carrier → Bool × Carrier
```

```
  ...        -- pairing and tagging laws only

structure Evidence
    (P : Pca) (C : ConstraintModel) where
  E : String → C.W → Set P.Carrier
  hered_E : ∀ {s w v}, C.Ri w v → E s w ⊆ E s v
  full_E  : ∀ {s w}, w ∈ C.F → ∀ x, x ∈ E s w
```

A partial applicative structure with total pairing and tag decoding — no combinatory completeness is assumed at this stage — and, for each atom and world, a set of elements evidencing that atom there. Evidence is hereditary ("belief increases along the branching order"; heredity of all the realisability relations below is a theorem) and full at fallible worlds, mirroring the fullness of the valuation there.

Three realisability relations are defined over this equipment. They agree at atoms (membership in the evidence set, or fallibility), at $\bot$ (fallibility), at $\wedge$ (pairing) and at $\vee$ (tagging), and differ only at $\twoheadrightarrow$ and $\bigcirc$ — and there only in *how much the realiser is given about the future*. Uniform realisability $\Vdash^{u}$ gives it nothing:

```
x ⊩ᵘ M ↠ N at w  iff  for all v ≥ w
                      and all b ⊩ᵘ M at v,
                      x·b is defined, ⊩ᵘ N at v;
x ⊩ᵘ ○M at w     iff  for all v ≥ w there is u
                      with v Rₘ u, x ⊩ᵘ M at u
```

— the same element must serve every future and every constraint-witness. Strategy realisability $\Vdash^{s}$, relative to an injective coding κ of worlds into the carrier, makes $\bigcirc$-evidence a function of the presented future:

```
| .somehow φ, x, w =>
    ∀ v, C.Ri w v → v ∈ C.F v
      (∃ y, P.app x (κ v) = some y ∧
        ∃ u, C.Rm v u ∧ P.fst y = κ u ∧
          realS P Ev κ φ (P.snd y) u)
```

In words: evidence for $\bigcirc M$ is a strategy which, handed the code of whatever future v has actually arrived, answers with a package naming a constraint-witness u — naming it, as κ u, rather than merely having one — together with evidence for M there. The strategy has no foreknowledge; the future is its input. The witness must be named because the elimination rule's computational content applies its continuation at that name; here the semantics tracks proofs, not merely truth.

Presented-strategy realisability $\Vdash^{p}$ does the same at implication: the realiser of $M \twoheadrightarrow N$ receives the pair $\langle \kappa\, v, b \rangle$ of the presented world and the argument:

```
| .ifThen φ ψ, x, w =>
    ∀ v, C.Ri w v → v ∈ C.F v
      (∀ b, realP P Ev κ φ b v →
        ∃ y, P.app x (P.pair (κ v) b) = some y ∧
          realP P Ev κ ψ y v)
```

One might suspect this of being a convenience. Section 5 shows it is forced.

5 The separations, and the obstruction

The three relations are separated on one three-world model — the split model: a root r below two incomparable leaves, A true at one leaf and B at the other — together with small variants.

Theorem 5.1 (the separations; `Belief/Realisability.lean`)**.** *(i) ([D18]) At the root of the split model $\bigcirc(A \vee B)$ is truth-forced, but no element $\Vdash^{u}$-realises it. (ii) ([D19]) An identity element $\Vdash^{u}$-realises $\bigcirc(A \vee B) \twoheadrightarrow (\bigcirc A \vee \bigcirc B)$ at every world of every model — a scheme PLL does not prove ([D20]). (iii) ([D21], [D22]) Under $\Vdash^{s}$ a case-splitting strategy realises $\bigcirc(A \vee B)$ at the root, and the distribution scheme acquires a countermodel. (iv) ([D23]) $(\bigcirc A \twoheadrightarrow \bigcirc B) \twoheadrightarrow \bigcirc(A \twoheadrightarrow B)$ is not $\Vdash^{u}$-valid.*

Read in order. A formula can be forced without being uniformly realised (i): a uniform realiser carries one tag, and would have to decide the disjunct before the branch occurs. Rigid evidence validates a strictly stronger logic (ii), which we call PLL^{u}; Section 10 identifies its natural candidate axiomatisation and what remains open. Strategy evidence supplies a realiser and restores the countermodel (iii), and is the first point in the development where evidence is literally a program. Uniformity also fails at implication (iv).

The natural question is then whether $\Vdash^{s}$ is the completeness-grade relation. It is not, and the reason is a theorem.

Theorem 5.2 (the obstruction; [D24], audit `[propext, Quot.sound]`)**.** *There is a three-world constraint model on which no evidence assignment whatever is simultaneously atom-adequate (evidence only where the atom holds) and full for $\Vdash^{s}$ (every forced formula has a realiser) — for every applicative structure able to implement finite lookup tables against the world-coding.*

```
theorem realS_fullness_obstruction
    (P : Pca) (κ : obsC.W → P.Carrier)
    (htab : ∀ g : obsC.W → P.Carrier,
      ∃ s, ∀ i, P.app s (κ i) = some (g i)) :
  ¬ ∃ Ev : Evidence P obsC,
      (∀ (a : String) (w : obsC.W),
        ∀ x ∈ Ev.E a w, w ∈ obsC.V a) ∧
      (∀ (φ : PLLFormula) (w : obsC.W),
        obsC.force w φ → ∃ x, x ⊩ˢ[Ev, κ, w] φ)
```

Note the quantifier structure: the evidence assignment is inside the negation. This is an obstruction, not a counterexample to one decoration. Since adequacy together with fullness is exactly the mutual induction any completeness-by-decoration proof must run — each consumes the other at the $\twoheadrightarrow$-clause — completeness of PLL for $\Vdash^{s}$ is not provable this way on these frames.

Proof. The model has a root below two incomparable leaves; an atom t holds at both leaves, p only at the first, q only at the second; the constraint relation is trivial. The proof is short, and we give it in full. Fullness at the atom t provides

tokens m_1, m_2 at the two leaves. One application of the table hypothesis then produces a *single* element s realising $\bigcirc t$ at both leaves simultaneously: presented κ(leaf), it answers with that leaf's own name and that leaf's token; and nothing about s records which leaf it is applied to, since a strategy is a function of the presented future and carries no world-marks. Now $\bigcirc t \twoheadrightarrow (p \vee q)$ is truth-forced at the root (vacuously there, since $\bigcirc t$ fails at the root; via the local atom at each leaf), so fullness provides a realiser x. The $\twoheadrightarrow$-clause of $\Vdash^{s}$ gives x the argument alone; instantiating it at each leaf with the same argument s, application being a function, both instantiations receive the same answer y — whose tag must be false at the p-leaf, since atom-adequacy has emptied the q-evidence there, and true at the q-leaf for the symmetric reason. This is a contradiction: a Boolean value cannot be both true and false. □

Two remarks. First, the obstruction is $\bigcirc$-essential. For a purely intuitionistic antecedent the argument collapses: world-tagged atom evidence would let a table-building structure case on the argument after all, and heredity ensures that forcing a disjunction at a branch point has already committed to a disjunct. What cannot carry world-marks is precisely a strategy, and that is the source of the obstruction to naive completeness. Second, the repair is thereby forced, not chosen: the $\twoheadrightarrow$-clause must be given the world, exactly as the $\bigcirc$-clause already is, and that single change is the definition of $\Vdash^{p}$. The separations survive the repair untouched, being stated for $\Vdash^{u}$ and $\Vdash^{s}$. This is the section's conclusion, and the paper's central claim: **what one may assume about how evidence meets the future determines which logic of belief one obtains.**

6 Completeness

In this section we prove completeness of PLL with respect to $\Vdash^{p}$-realisability. The proof has two independent halves — a decoration theorem, saying that every suitable countermodel refutes in the realisability sense, and a countermodel-existence theorem — and we take them in that order. Both halves are effective in a sense made precise below; the existence half finitises the canonical construction of [24, §4], replacing its one appeal to Zorn's lemma [70] with an iterated appeal to the mechanised decision procedure.

6.1 Decoration: adequacy and fullness

The frames to be decorated are *finite models as data*: the structure `FinCM` [D25] (`PLL/Semantics/CountermodelEmit.lean`) packages worlds 0,…,n−1, the two relations as lists of pairs, the fallible set and the valuation as lists, together with an executable forcing function `forceB` [D26] and a verified reflection lemma ([D27]) identifying the computation with genuine forcing in the induced constraint model. On such a frame, *token evidence* gives each atom one designated element, exactly where the atom holds. All the evidence a completeness proof needs, it turns out, is one token per atom.

Theorem 6.1 ([D28], audit `[propext, Quot.sound]`)**.** *Over any checked finite frame with token evidence, and any applicative structure with two table-forming capacities against the world-coding: for every formula φ,*

```
(∀ w x,
   (x ⊩ᵖ[tokenEvidence P M h tok, κ, w] φ) →
   (M.toModel h).force w φ) ∧
(∃ wit : Fin M.n → P.Carrier, ∀ w,
   (M.toModel h).force w φ →
   (wit w) ⊩ᵖ[tokenEvidence P M h tok, κ, w] φ)
```

— whatever is realised is forced (adequacy), and whatever is forced is realised by an explicit per-world witness function (fullness).

The shape of the fullness clause deserves a remark before the proof: not "at each world there is a witness" but "there is a function of worlds". The induction requires the whole function at once, because the witnesses at $\twoheadrightarrow$ and $\bigcirc$ are tables whose entries are the subformula's witnesses at other worlds.

Proof. By a single induction on φ proving the conjunction; each case may use both halves of the inductive hypotheses, and the cases at $\twoheadrightarrow$ exhibit the mutual dependency in both directions. Throughout, the reflection lemma converts freely between forcing and its executable form, and this is what makes the theorem constructive: every branch decision in a witness is a computation, not a case split on an undecided proposition.

At atoms, $\bot$, and $\wedge$ there is nothing surprising. At $\vee$, adequacy reads the realiser's tag, and the fullness witness *computes* the branch: it runs the executable forcing of the left disjunct at w and tags accordingly — no excluded middle is consulted; the decision procedure for forcing is. At $\bigcirc$, adequacy is a direct reading of the strategy's answers, and fullness tabulates a bounded, decidable search of the finite frame for a constraint-successor forcing the body, which the $\forall\exists$ clause guarantees to succeed. (Both witness computations are given in full in Appendix C.)

At $\twoheadrightarrow$, the case where the obstruction is answered. Adequacy: given a realiser x of $\varphi \twoheadrightarrow \psi$ and a future v forcing φ, feed x the pair $\langle \kappa\, \mathrm{v}, \mathrm{wit}\varphi\, \mathrm{v}\rangle$ — the antecedent's fullness witness — and apply ψ's adequacy to the answer. Fullness: apply the pair-table capacity to ψ's witness function, obtaining a single element s with $\mathrm{s}\cdot\langle \kappa\, \mathrm{v}, \mathrm{b}\rangle = \mathrm{wit}\psi\, \mathrm{v}$ for every v and b. The obligation fires only when b realises φ at v; then φ is forced at v by φ's adequacy, so ψ is forced at v since $\varphi \twoheadrightarrow \psi$ is forced below, and $\mathrm{wit}\psi\, \mathrm{v}$ realises it by ψ's fullness. The table ignores its second argument entirely: the presented world alone determines the right answer — which is precisely the lookup Theorem 5.2 proves impossible when the world is not presented. □

From Theorem 6.1 and soundness one obtains at once the *refutation lemma* ([D29]): every countermodel validated by the verified checker is a $\Vdash^{\mathrm{p}}$-refutation of its sequent — hypotheses realised at the refuting world, conclusion unrealisable. Both table capacities are discharged, with nothing assumed, by an explicit algebra of finite lookup tables (`Tbl` [D30], `tblPca` [D31]: base codes, formal pairs and tags, two table formers; application is decidable lookup; the laws hold by definitional computation), so the refutation lemma has a closed, hypothesis-free form ([D32]).

6.2 The countermodels, finitely and without Zorn

It remains to produce a checked countermodel for every underivable sequent. Here the paper rejoins its parent most closely, and the reader may find it useful to have [24, §4] open beside this subsection.

The canonical model there is built from worlds which are *triples* (Γ, Δ, Θ) of sets of formulas — validated, falsified, and falsified at every R_m-reachable world — the third component being, as the original explains, a special feature introduced because the falsity of a formula $\bigcirc M$ cannot be recorded by placing M, or any subformula of M, in the other two components; it must be tracked separately. Under reading (7) the third component is the record of a *promise*: a formula the believer has pledged never to evidence, however the constraint is discharged. Promises are what prevent the fallible world, which validates everything, from trivialising the $\forall\exists$ clause. A triple is consistent if for no finite selections $N_1,\ldots,N_n$ from Δ and $K_1,\ldots,K_k$ from Θ, jointly nonempty, does

$$\Gamma \vdash N_1 \vee \cdots \vee N_n \vee \bigcirc(K_1 \vee \cdots \vee K_k)$$

— note the $\bigcirc$ guarding the promises, the key subtlety of the construction: refuting a promise-selection means deriving somehow-one-of-the-promised, which is exactly the form that $\bigcirc$-elimination can act on. The original then takes *maximal* consistent triples, by Zorn, and proves a truth lemma.

The present construction (`PLL/Semantics/FinComp.lean`) keeps every one of these design decisions and changes exactly two things: the triples are finite (`FTheory` [D33]: three finite sets, and the classical consistency notion is reused verbatim through the evident coercion, so the entire lemma library about the consistency formula transfers without re-proof); and maximality is replaced by *totality over the subformula closure* of the sequent under refutation. The replacement is justified by two observations.

The first observation ([D34]) is that consistency of a finite triple is *one derivability question*: since the consistency formula is monotone in both selections, the universally quantified condition collapses to its worst case, the full selections. That single question is decided by the mechanised decision procedure. Consistency of finite triples is decidable — and so the Lindenbaum construction can be a computation:

```
noncomputable def extendStep
    (T : FTheory) (φ : PLLFormula) : FTheory :=
  if (T.insVal φ).Cons
  then T.insVal φ else T.insFal φ
```

Walk the closure once; at each formula ask the decision procedure whether believing it preserves consistency; believe if so, refute if not. The else-branch is safe by the extension dichotomy ([D35]): if both extensions were inconsistent, the two failure witnesses would combine — by the same manipulation of selections as in the original's maximality lemma — into an inconsistency of the triple one started with. The result ([D36]) is a closure-total consistent extension *with the promises untouched*: the fold never writes to the third component, an equality the truth lemma uses twice.

The second observation is that under totality the original's maximal-theory lemmas — deductive closure, primeness of Γ, the decomposition laws for falsified conjunctions, disjunctions and implications, $\Theta \subseteq \Delta$ — all *simplify*. Maximality entered those proofs only through extension-failure witnesses, existentially quantified selections needing combination; under totality every witness is a singleton. Primeness, in full: if $M \vee N$ is validated but neither disjunct is, totality refutes both, and the selection {M, N} together with $\vee$-elimination contradicts consistency directly. Five lines of formal text against the original's thirty. Constructivisation, for once, is a simplification.

Theorem 6.2 (truth lemma; [D37])**.** *On the model whose worlds are the closure-total consistent triples, with R_i inclusion of validated parts, R_m inclusion of validated and promised parts, the fallible worlds those validating $\bot$, and atoms outside the closure true everywhere: for every closure formula φ, membership in the validated part forces φ, and membership in the falsified part refutes it.*

The propositional cases follow from the simplified lemma suite. Three cases require a Lindenbaum extension, and they are the exact three of the original, each start-triple carried over unchanged; only the construction that extends them has changed. To refute a falsified $M \twoheadrightarrow N$ one extends $(\Gamma \cup \{M\}, \{N\}, \varnothing)$. To force a validated $\bigcirc M$ at an arbitrary future one extends $(\Gamma_1 \cup \{M\}, \varnothing, \Theta_1)$ — the body added, the *promises carried across* — and consistency of this triple is where the $\bigcirc$-guard does its work: a violation would give $\Gamma_1 \vdash M \twoheadrightarrow \bigcirc(\bigvee$ of a Θ_1-selection), whence, since $\bigcirc M$ is validated at the future, the elimination composite yields $\Gamma_1 \vdash \bigcirc(\bigvee$ of the selection), an inconsistency of the future with its own promises. The extension is then a constraint-successor — beliefs grew, promises exactly preserved — forcing M. To refute a falsified $\bigcirc M$ one extends $(\Gamma, \varnothing, \{M\})$: a promise *created*. Its consistency reduces, by the monotonicity composite for $\bigcirc$, to $\Gamma \vdash \bigcirc M$, contradicting the falsification; and every constraint-successor of the extension inherits the promise, hence — promises being falsified, by the $\Theta \subseteq \Delta$ law — refutes M. So the $\forall\exists$ clause fails at the extension, which is an ordinary future of the world one started at. This is the semantic role of promises;

the executable countermodels of Section 7 display the same construction.

Corollary 6.3 ([D38], [D39]). *Every underivable sequent is refuted at a world of its finite canonical model; enumerating the (finitely many) worlds yields the countermodel as literal checker data:*

```
¬ (Γ ⊢ C)  implies  ∃ M w, checkB M w Γ C = true
```

Consider what has been achieved axiomatically. No maximality principle appears anywhere above, and after the axiom-hygiene pass the entire chain of this subsection — the decidability of consistency, the decided Lindenbaum fold (now, in fact, a computable function), the truth lemma, and the countermodel enumeration — audits at `[propext, Quot.sound]`: no choice (what little classicality survives in the artefact is a statement-vocabulary matter, discussed in Section 9). It is tempting, and would be wrong, to say that the logic proves its own completeness: the proof is carried out in the metatheory, and a quantifier-free propositional logic cannot even state completeness. What is true, and we think worth saying carefully, is that the metatheoretic proof's only oracle is the object logic's own decision procedure — an arrangement made possible by the fact, special to a decidable logic with no embedded arithmetic, that there is no Gödelian obstruction to consulting it.

Remark 6.4 (the apparent circle)**.** Decidability before completeness has an air of circularity, since the familiar route runs the other way: the original paper's Theorem 2.8 obtained decidability *from* completeness, through the finite model property, and a decidability so obtained could not be used inside a completeness proof without vicious regress. The present arrangement is innocent, for three reasons. First, the decision procedure's correctness is a fact of proof theory alone — the repaired terminating calculus proves exactly the PLL sequents, and the loop-checked search decides that calculus; no model is mentioned anywhere in the chain. Second, the completeness proof consumes decidability only as a *derivability* oracle, never a semantic one: what the oracle contributes is the totality of the case analysis "believe or refute". The classical proof settles this by Zorn's lemma and excluded middle; here it is settled by a computation. The semantics enters only afterwards, in the truth lemma. Third — an argument available only to a mechanised development — the absence of the circle is itself machine-checked: the import graph is acyclic by construction, the completeness modules import the decidability module, and the decidability module imports no semantic module whatever. The reversal has a respectable pedigree: decidability of intuitionistic propositional logic [31] preceded its Kripke completeness [43] by thirty years, and for the same reason: a terminating calculus is a purely syntactic object. Completeness then shows that this syntactic calculus suffices to build every countermodel; the dependency runs in one direction only, and the module import graph enforces it.

6.3 The theorem

Theorem 6.5 ([D40]). *A sequent is derivable in PLL exactly when no token-decorated, checker-verified finite model $\Vdash^{\mathrm{P}}$-realisability-refutes it:*

```
Nonempty (LaxND Γ C) ↔
  ¬ ∃ (M : FinCM) (w : Nat)
      (hwf : M.WellFormed) (hlt : w < M.n),
    (∀ ψ ∈ Γ, ∃ x,
       x ⊩ᵖ[Ev₀, tblK, (w, hlt)] ψ) ∧
    ¬ ∃ x, x ⊩ᵖ[Ev₀, tblK, (w, hlt)] C
  -- Ev₀ := tokenEvidence tblPca M hwf tok₀
```

Forward: soundness together with Theorem 6.1 — a derivable conclusion is forced wherever its realised hypotheses are, hence realised by its own fullness witness. Backward: Corollary 6.3 composed with the closed form of the refutation lemma. Every quantifier ranges over concrete objects — finite models as printable data, one token per atom, realisers in an explicit algebra of lookup tables — and the backward case decision is made by the decision procedure, not by excluded middle. The theorem audits at `[propext, Quot.sound]`, enforced by build-time guards: the completeness of PLL for realisability is, in the formal sense the audit measures, a choice-free theorem. PLL is complete for the semantics in which evidence may react to the future; incomplete, by Section 5, for every more rigid discipline.

7 The theory, run

(This section's artefacts are all generated: countermodels by evaluation of the emitter, proof terms by the kernel; each is re-checkable from the cited files.)

The completeness theorem's countermodels are not merely finite but executable, and the artefact exploits this in both directions. An untrusted search procedure (`CounterEmit.emit` [D41]) proposes countermodels from failed proof search; the verified checker certifies them; a wrong proposal can produce only a failure, never a wrong certificate. In the other direction, any certified model refutes in the full realisability sense, by the refutation lemma.

That discipline is packaged as a single entry point. `PLL/Search/Search.lean` attempts a sequent *either way* — a certified battery of small frames and the closure emitter for refutations, a fuel-free backward searcher over the terminating calculus for proofs — and returns a verdict carrying its own certificate: a proof term the typechecker validates, or a finite model the verified checker accepts. The third verdict is an explicit `unknown`, which asserts nothing. Soundness is therefore structural rather than a property of the search code: a wrong internal guess can only degrade a verdict to `unknown`, never manufacture a false one. This is the development's practical engine, and the division of labour is deliberate. The

decision procedure behind Remark 6.4 is constructive and total, but its fuel bounds are far too large to run on any but trivial sequents; proof extraction and automation therefore use the searcher and keep only its certificates. The cost profile is recorded honestly alongside it: successes are cheap, the worst case is exponential — provability is already PSPACE-hard for the ○-free fragment — and the frame battery is incomplete by design, a first filter rather than a complete refuter.

For $\bigcirc p \vdash$ p the emitter produces a five-world model, pinned in the artefact as data (`demoM` [D42]), whose worlds read as belief states: a state believing exactly $\bigcirc p$ (the refuting world); above it a state where the constraint has been discharged and p is believed; the fallible top world; and a state that has *promised* never to evidence p, whose only constraint-successor is itself. The kernel-checked certificate

```
theorem somehow_p_not_p :
    [○(prop "p")] ⊬ prop "p" :=
  FinCM.not_provable_of_check
    (M := demoM) (w := 2) (by decide)
```

runs the verified checker inside the kernel; no compiled evaluation is trusted, and the audit is choice-free. Under reading (7), believing $\bigcirc p$ cannot be converted into believing p, and the countermodel shows why: there is a coherent belief state in which the constraint's discharge remains forever pending. Figure 1 is the model, drawn by the artefact from its own data.

For the distribution scheme $\bigcirc(p \vee q) \vdash \bigcirc p \vee \bigcirc q$ the emitter builds twenty worlds, and its report shows the promise machinery of Theorem 6.2 directly:

```
refuting world: w4  (20 worlds)
  w4: believes [○(p v q)]
  w5: believes [○(p v q)], promises-false [○p, p]
  w6: believes [○(p v q)], promises-false [○q, q]
  ...
```

The refuting world believes exactly $\bigcirc(p \vee q)$; its two critical futures carry the standing promises "never p" and "never q"; a promise binds all constraint-successors, so $\bigcirc p$ fails along one future and $\bigcirc q$ along the other, while each future can still discharge the *disjunction* through its unpromised disjunct. The twenty worlds are an artefact of the systematic construction — the canonical enumeration keeps every coherent belief state — and a checker-gated *minimisation* removes redundant worlds: repeatedly delete any world whose removal leaves the certificate intact (`emitMin` [D43]; untrusted, like the emitter, and harmless for the same reason). For this sequent the minimiser descends from twenty worlds to three — a root believing $\bigcirc(p \vee q)$ below one leaf satisfying p and one satisfying q, with $R_m = R_i$ and neither promises nor a fallible world remaining — which is *precisely* the hand-built split model of [24, Fig. 3], now pinned as a build-time regression guard. The systematic model proves existence; deletion recovers the presentable one; and a failed proof search terminates in a

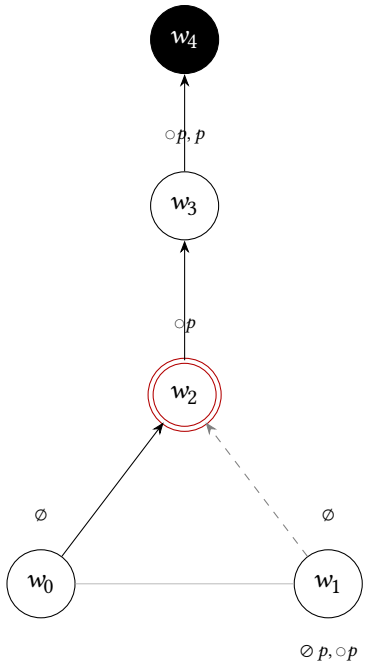


Figure 1. The emitted five-world countermodel to $\bigcirc p \vdash p$ (`demoM`). Labels above a world are its beliefs, labels below its standing promises (∅ marks a formula promised never to be evidenced); the red doubled node is the refuting world, the black node the fallible top world. Solid arrows are constraint steps (R_m), dashed grey arrows information steps (R_i only), and the thin grey edge joins the mutually R_i-related pair of Remark 7.1. Generated by `PLL/Search/Diagram.lean`.

structured epistemic explanation rather than a bare verdict. A second, finer pass (`emitMinClean` [D44]) sweeps attributes rather than worlds, deleting any fallible flag, valuation pair or relation pair the certificate survives without; it settles finer questions — one constraint arrow of the split model does no work, and the countermodel to $\bigcirc p \vdash$ p needs no fallible world at all, minimising to two worlds: believe nothing, then p. Both prunings are pinned, and Figure 2 shows the first.

Remark 7.1 (preorder, not partial order)**.** In all these models R_i is a genuine preorder: distinct worlds can be mutually related. The twenty-world model has thirteen such pairs, among them the refuting world and its two critical futures — w_4, w_5, w_6 believe exactly the same formulas and differ only in their standing promises, so the refutation just displayed takes place entirely inside a single R_i-equivalence class. This reflects the construction: canonical R_i compares beliefs and deliberately ignores promises; refuting a ○-formula changes no belief — it only adds a commitment. R_m, which carries promises along, is antisymmetric in every model of this section, and nothing is lost for validity: forcing is hereditary along R_i ([D45]), minimisation removes the equivalent twins, and both minimised models above are posets (build-time

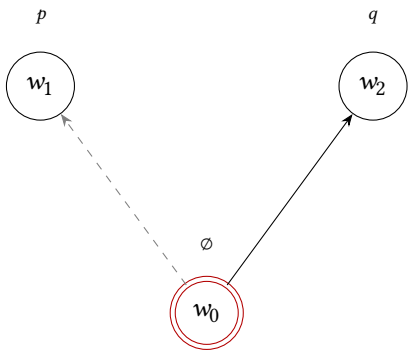


Figure 2. The minimised countermodel to $\bigcirc(p \vee q) \vdash \bigcirc p \vee \bigcirc q$ after attribute-cleaning: the root (refuting, red) reaches its constraint-witness through the q-leaf alone — the solid R_m arrow — while the edge to the p-leaf is an information step only (dashed grey); the redundant constraint arrow of the hand-built split model has been deleted by the checker-gated sweep. Generated by `PLL/Search/Diagram.lean`.

guards `riEquiv` [D46]/`rmEquiv` [D47]). One design observation, recorded without mechanised warrant: a canonical R_i refined to respect promises — antisymmetric by construction — would obstruct the $\bigcirc$-case of the truth lemma, whose refuting step needs a same-belief future in which the old promises have lapsed and one fresh promise stands. The preorder is therefore not an accident of the construction but is required by the $\bigcirc$-case of the truth lemma.

Finally, evidence in the positive direction. The sequent $\bigcirc A, \bigcirc(A \twoheadrightarrow B) \vdash \bigcirc B$ — combine a belief with a believed implication — has the kernel-checked proof term

```
let val• := #0 in (let val• := #2 in val (#0 #1))
```

two monadic binds and an application: run the belief in A, obtaining evidence under its constraint; under that constraint run the belief in $A \twoheadrightarrow B$; apply; return under the composed constraint. The uniform extraction of the artefact ([D48]) compiles this derivation, by the equations "laxIntro is the identity, laxElim is abstraction-then-application", to the polynomial

```
(λa. (λg. g · a) · f) · x
```

whose value $\Vdash^{\mathrm{u}}$-realises the conclusion wherever the context is realised; the strategy extraction compiles the same derivation to the reactive form in which the presented future is fed to the first strategy, the named witness it returns is where the second strategy is run, and the results compose. Sequential composition of beliefs is the meet law of the evidence semantics ([D49]), and the artefact records — one of the development's early machine-checked findings — that no confluence assumption appears anywhere: belief composition is sequential, not confluent.

A second example, this time negative. Idempotence $\bigcirc\bigcirc M \dashv\vdash \bigcirc M$ is two proof terms: the multiplication $\mu = \lambda$x. let val $y \Leftarrow x$ in y and the unit $\eta = \lambda$x. val x. Are they mutually inverse?

Proposition 7.2 ([D50], [D51]; `PLL/ND/Idempotency.lean`)**.** *$\mu \circ \eta$ reduces to the identity; $\eta \circ \mu$ is not even convertible to it. Idempotence is inter-derivability, not isomorphism.*

The first composite is the monad's unit law, holding as computation (the let fires on the val); the second normalises to λx. val (let val $y \Leftarrow x$ in y), a stuck term in which the collapse remains pending inside the re-assertion, and the artefact computes the composite's complete reduction graph — four terms, closed under reduction, the identity not among them. The separation holds in the strongest sense the calculus can express: the reduction is confluent (`PLL/Normalisation/Confluence.lean` — Newman's lemma [52] from strong normalisation, the critical pairs being `let`-β and `let`-assoc overlapping at a `bind`), so normal forms are unique, conversion is decided by the normaliser, and the failure is of convertibility, not merely of reduction. Read doxastically, collapsing a belief about a belief and then re-asserting belief in the result is not a null operation on evidence, though it is one on truth.

One asymmetry visible in these examples is worth noting. Evidence extracted from proofs is *frame-uniform*: one polynomial serves every model. The fullness witnesses of Theorem 6.1 are *frame-aware*: tables built by surveying the finite frame. Provable formulas have evidence needing no knowledge of the situation; merely true ones have evidence only relative to a survey of this situation's futures. The idealisation Hintikka identified appears exactly in the completeness theorem's omniscient survey: stated explicitly, and confined to where it belongs.

Finally, a first applied step from the programme that motivated PLL [23]. Mendler's timing analysis of combinational circuits [50] reads a proof as a stabilisation bound — unit: a wire, delay 0; multiplication: sequential composition, +; strength: reconvergent merge, max — and the artefact evaluates PLL proof terms in exactly this delay algebra (`PLL/Timing/Timing.lean`, `PLL/Timing/TimingAdder.lean`, `PLL/Timing/TimingRipple.lean`, `PLL/Timing/TimingLookahead.lean`). The reconvergent-fanout circuit of [50, §7] is reproduced with its false-path result — on the input class holding a = 1 the extracted bound is strictly below the topological longest path, because a proof traverses only real logical dependencies — and a four-bit carry-skip adder is certified faster than its topological bound ([D52], [D53]; both with *no axioms at all*). These false-path results are one side of a dichotomy the artefact now makes precise. Extraction never exceeds the topological longest path; the interest is in when it falls strictly below. The carry-ripple adder is the tight extreme, and it is now treated at *every* width: one reusable carry cell applied n times to the carry-in — so that the proof term is literally the cell iterated n times — has extracted delay exactly $n \cdot \delta$, by induction on the width ([D54]), and this *equals* the topological path

([D55]), a single carry chain having no false path to discard. With the carry-skip adder strictly below, the two *bracket* the general fact — proof-term extraction is at most topological analysis, with equality exactly when the longest path is sensitisable. The same induction exhibits the extracted delay as the nesting depth of the carry cell in the proof: critical-path length is proof depth.

That identity turns circuit optimisation into proof restructuring, and the artefact certifies the paradigm case. A carry-lookahead adder produces the same group carry as the ripple by folding the per-bit generate/propagate pairs under their associative merge as a *balanced tree* rather than a line. Abstracting a pair as one lax signal and the merge as a single reusable cell, the balanced fold of depth k and the ripple-shaped fold of length $2^k - 1$ are two proof terms of the *same* sequent built from the *same* $2^k - 1$ merge cells; re-association alone drops the delay from $(2^k - 1) \cdot \delta$ to $k \cdot \delta$ — linear to logarithmic in the leaf count, with no extra hardware ([D56]). That the two networks compute the *same* carry is proved, not assumed: the extractor is parametric in its value layer, so running the same two proof terms with the carry pair in place of the trivial layer returns the same group generate/propagate pair, for any associative merge and hence for the carry algebra itself ([D57], [D58], with the merge's associativity checked axiom-free); a single evaluation yields both that value and the delay. Ripple is the deepest proof of the group carry, lookahead a shallow one, and the speed-up is precisely the reduction in $\bigcirc$-nesting depth — a re-association of the proof, not a change in what is proved.

The two remaining circuits of the case-study list are asynchronous, and between them they display one law of the modality that holds and one that fails. Muller's *C-element* waits for both of its inputs; logically it is the meet law $\bigcirc A \wedge \bigcirc B \vdash \bigcirc(A \wedge B)$, which is derivable. Its canonical derivation is sequential — two binds, so the multiplication charges $\delta_a + \delta_b$ — while the element realises the same law through the strength, charging $\max(\delta_a, \delta_b) + \delta_C$. The element is thus a cheaper realisation of a derivable law, and it pays exactly when $\delta_C < \min(\delta_a, \delta_b)$ ([D59]). An *arbiter* grants one of two competing requests. Under contention it believes that some grant occurs, $\bigcirc(g_1 \vee g_2)$, while believing of neither grant that it occurs, since nothing in the inputs determines which way the resolution falls; that is exactly the failure of distribution over $\vee$, and it is refuted by a three-world model read as the arbiter itself ([D60], choice-free). The reason is structural: mutual exclusion means the two grant-futures never reconverge, so the frame is not mutually confluent ([D61]), and mutually confluent frames are exactly the models of PLL with the distribution scheme ([D62]). That the arbiter may not grant twice is the circuit-level reason the scheme fails. The model is the split model of [24, Fig. 3] once more, now with a reading: contention below, the two resolutions above. Under reading (7) these are all belief computations: a signal's delay is the cost of discharging the constraint under which its stability is believed, and re-associating a proof of equal size into a shallower one is how that cost is cut.

We are explicit about what this is and is not: it is the delay-algebra and proof-theoretic half of the programme's first case study — extraction and refutation in the constraint semantics, not in the evidence semantics of Sections 4–6. The study itself — these same netlists with their stabilises-to specifications formalised *in the evidence semantics*, the components composed into a larger netlist under a formally combined specification, $\bigcirc$(out = spec) proved there and its evidence term extracted — requires a realiser layer that is designed but not implemented; a staged design brief accompanies the artefact, and the programme — timing at scale, proof-carrying authorization, certified effectful computation — is the subject of a companion paper in preparation.

8 Related work

The nearest neighbour of reading (7) in the literature is the intuitionistic epistemic logic of Artemov and Protopopescu [6], whose belief system IEL$^-$ adopts co-reflection $A \twoheadrightarrow KA$ and rejects factivity, so that $K\bot$ is consistent, exactly as $\bigcirc\bot$ is here. PLL adds idempotence; we conjecture PLL-$\bigcirc$ is exactly IEL$^-$ extended by $\bigcirc\bigcirc A \twoheadrightarrow \bigcirc A$, and record the question as open. IEL itself has been mechanised, in Coq [33]. Justification logic [3–5] in its forgetful reading forces factivity and is therefore not this.

Two classical traditions are illuminated by the collapse theorem. Doxastic logic in the Hintikka–KD45 tradition [21, 34, 60] is the baseline against which Proposition 3.4 should be read: the lax believer is fully introspective but has no D: an inconsistent belief is admitted here, where KD45 forbids it. The evidence models of the classical belief-revision literature [8, 9, 67] build belief from evidence with interior-type operators over a *topology*; Theorem 3.1 says why that extra structure is obligatory — on the bare classical algebra every nucleus is closed, so evidence must live somewhere else. And a security literature arrived at the two signature facts of Section 3 independently, in doxastic vocabulary: in the authorization logics descended from the dependency core calculus [1, 29], the modality "K says" validates $A \twoheadrightarrow$ (K says A) — every principal believes what is true — and (K says $\bot$) does not entail $\bot$: a principal may assert the absurd without collapsing the logic. A lax modality read as an agent's attitude — truth-closed, and admitting an inconsistent belief without collapse — is already used in engineering practice.

The constraint reading of the parent paper and the monadic reading of computational type theory [11, 51] are the two established computational faces of $\bigcirc$; the doxastic reading is compatible with both, and the double-negation believer — in constraint models with $R_m = R_i$ and no fallible worlds, $\bigcirc M$ is forced exactly where $\neg\neg$M is (machine-checked, [D63]) — recovers the continuation

reading [44] as a special case. On the mechanised side, the normalisation-by-evaluation line for modal and lax calculi in Agda [65, 66] is the closest packaged artefact to our term-calculus layer, and the relation is technical, not merely bibliographical. That semantics interprets the lax modality by a *bare existential* over the constraint relation — a clause that is not hereditary by itself — and accordingly assumes *forward confluence* of the two accessibility relations; the parent paper's ∀∃ clause internalises heredity and assumes nothing. The two clause styles are reconciled exactly on the mutually confluent frames of Section 10: there the ∀∃ clause collapses to the bare existential ([D64]), and the logic of that frame class is precisely PLL + ○(A ∨ B) ↠ (○A ∨ ○B) ([D62]) — a scheme PLL does not prove. The simpler clause is therefore bought at the price of a strictly stronger logic. Nothing in the evidence semantics of Sections 4–6 assumes confluence anywhere; in the one place it might seem needed — the meet law composing two beliefs — the mechanised proof is sequential composition, discharging the second constraint at the world the first produced (Section 7), never a common refinement of independent successors. We are not aware of a prior completeness theorem for a lax logic with respect to a realisability semantics, nor of a prior mechanised canonical-model construction whose Lindenbaum step is an appeal to the object logic's decision procedure. The terminating-calculus route to decidability for PLL is due to Iemhoff [38], resting on a general G3i-to-G4i transfer [37], with earlier proof search for PLL due to Howe [35]; the artefact contains a machine-checked counterexample to the published calculus and a repaired one, communicated separately.

9 Method

Every claim of this paper is a theorem of a Lean 4 [16] development [22], over Mathlib [61], that also contains its parent paper entire. The axiom footprint of each result is recorded: nothing at all for a handful of results (among them a well-foundedness theorem for the Kleene–Brouwer order [40], a standalone fully constructive result in the artefact); `[propext]` or `[propext, Quot.sound]` — no choice — for the proof theory, the decoration half of completeness, the obstruction, and the executable countermodel certificates, the decidability of the logic, the finitised canonical model, and the completeness theorem itself; classical axioms remain only in the parent's Zorn completeness (kept deliberately, as the foil) and in a handful of statements whose Mathlib vocabulary (`List.toFinset`, `Finset.toList`, the lattice ∪/∩) is choice-tainted at definition level — an audit walks the bodies of the constants a *statement* mentions, so such a phrasing can never audit clean, however its proof is conducted; each is a one- or two-line wrapper around a clean restatement through a choice-free finite-set toolkit.

Two remarks on what the method made feasible. The axiom-hygiene pass itself — re-founding a completed completeness proof choice-free, by replacing a classical finite-set vocabulary, a classical decidability instance and a classical tactic with constructive carriers throughout a multi-thousand-line development — is the kind of change that would be hard to justify, or to referee, done by hand; with the proof assistant tracking the invariants it was routine. And one metatheoretic result appears to be new to the mechanised literature in passing: the strong normalisation of Section 2 is, as far as we can determine, the first mechanised ⊤⊤-lifting with sums — the sole prior mechanisation of strong normalisation for the computational metalanguage treats the core calculus without sums, in Isabelle/HOL-Nominal [17], and the paper-form treatments of sums [45, 46] are unmechanised; cf. the POPLMark Reloaded benchmark [2] for mechanised logical-relations normalisation.

We have already said that the formalisation functioned as an instrument. It is worth saying how. Three intended theorems failed, and their failures were themselves established: a collapse lemma for $\Vdash^s$ (refuted; the reformulation became the refutation lemma), the claim that ○ is not a normal modality (refuted; it is normal, every nucleus validating K), and the intended completeness of $\Vdash^s$, whose failure is Theorem 5.2 and whose repair is the paper's central definition. A referee may reasonably ask what in this paper was found by the human author and what by the machine; the honest answer, recorded commit-by-commit in the artefact's history, is that the direction, the reading, the standards and the corrections were human, and a large part of the construction, the drafting and two of the three failures were found by a language model operating the proof assistant under those standards. Section 9.1 states this precisely.

9.1 Disclosure of AI assistance

The formalisation and the drafting of this paper were carried out in an extended collaboration between the author and a large language model (Anthropic's Claude — Opus 4.8 and Fable 5 — operating the Lean toolchain directly). The model wrote the majority of the Lean proof scripts and of the prose here, proposed constructions, and in several instances discovered the mathematical turning points — notably the fullness obstruction of Section 5, found in the attempt to prove its negation. The author set the research programme and the doxastic reading, fixed the methodological standard — no claim asserted beyond its machine-checked status, with axiom audits recorded per theorem — directed every design decision, rejected and corrected the model's errors, and takes sole responsibility for the mathematics and the text. Under prevailing authorship norms a language model cannot be an author; its contribution is recorded here and, commit by commit, in the artefact's history. The kernel-checked artefact is indifferent to the provenance of its proofs; the reader need

trust neither the author nor the model, which is the point of the method.

On routes to publication for machine-checked mathematics by unaffiliated authors, and the artefact-citation norms this paper relies on (Zenodo-archived repository, `CITATION.cff`) — with the Lean formalisation of perfectoid spaces [12] an established precedent for citing a repository as a refereed artefact — see the survey in the artefact (`docs/publication-routes.md`).

10 Conclusion and open problems

However strange a belief modality without factivity may at first appear, it is a natural object, known already from half a dozen mathematical contexts; what the present paper adds is that its logic of evidential belief is non-degenerate exactly over intuitionistic logic, and that PLL is precisely complete for the evidence discipline in which realisers meet the future as an input. The countermodels witnessing the "precisely" are finite, executable, and readable as belief states; the canonical construction that produces them is the parent paper's, with Zorn's lemma traded for the logic's own decision procedure.

Open problems. The identification of $\mathrm{PLL}^{\mathrm{u}}$, the logic of rigid evidence — for which the natural candidate axiomatisation is the distribution scheme itself. The parent paper's Theorem 4.5 ties $\bigcirc(A \vee B) \twoheadrightarrow (\bigcirc A \vee \bigcirc B)$ to *mutual confluence* of the two accessibility relations — from any world, an R_m-step and an R_i-step close at a common world — under which the $\forall\exists$ clause collapses to bare possibility, so that credibility, once attained, is already stable. The artefact now carries both sides. Soundness: the collapse ([D64], no axioms at all), the validity of the scheme on confluent models ([D65]), and the fact that the split model refuting the scheme fails confluence exactly ([D66]). And completeness, proved here: *PLL extended by the scheme — taken as additional hypotheses — is sound and complete for mutually confluent constraint models* ([D62]), by a canonical model whose worlds are the deductively closed prime sets of formulas, in which $\bigcirc^{-1}$ of a prime set is prime *precisely because of the scheme*, and mutual confluence holds with $\bigcirc^{-1}$ of the R_i-successor as witness; no promise component is needed anywhere, because on confluent frames the clause is already bare possibility. What remains open is the realisability side of the identification: whether $\Vdash^{\mathrm{u}}$-validity coincides with validity over confluent constraint models — that is, whether $\mathrm{PLL}^{\mathrm{u}}$ is precisely PLL + $\bigcirc(A \vee B) \twoheadrightarrow (\bigcirc A \vee \bigcirc B)$. The exact relation of PLL-$\bigcirc$ to IEL^{-} (conjecturally: idempotence). A Kleene-algebra instantiation of the table capacities via partial computable functions [41] (the choice-free statement needs care around partiality). The tripos-level formulation of the evidence semantics [13, 36, 69]. Uniform interpolation for PLL — ordinary Craig interpolation is settled, choice-free, in the artefact (Section 2), while the uniform case is where the repaired terminating calculus supplies the height bound the published method lacked [56]; the mechanised investigation continues, with the Coq/Rocq mechanisations of uniform interpolation for intuitionistic and intuitionistic modal logics [27, 58, 68] as the reference points, alongside a live Lean interpolation effort for PDL [30].

A The Mechanised Base

This appendix reproduces, verbatim from the artefact, the definitions a reader must accept as formalising the intended notions, and the exact statements of the audited theorems of Section 2.

A.1 Language and natural deduction

```
inductive PLLFormula where
  | prop (constantName : String)
  | falsePLL
  | and (a b : PLLFormula)
  | or (a b : PLLFormula)
  | ifThen (antecedant consequent : PLLFormula)
  | somehow (a : PLLFormula)   -- the lax modality ○
```

`notPLL`/`truePLL` are the defined $\neg/\top$. Contexts are lists; the identity rule is membership-based, so weakening, exchange and contraction are admissible (`LaxND.rename`), not structural:

```
inductive LaxND : List PLLFormula → PLLFormula → Type
  | iden      {Γ φ}   (h : φ ∈ Γ) : LaxND Γ φ
  | falsoElim {Γ} (φ) (p : LaxND Γ .falsePLL) :
      LaxND Γ φ
  | impIntro  {Γ φ ψ} (p : LaxND (φ :: Γ) ψ) :
      LaxND Γ (.ifThen φ ψ)
  | impElim   {Γ φ ψ} (p₁ : LaxND Γ (.ifThen φ ψ))
      (p₂ : LaxND Γ φ) : LaxND Γ ψ
  | andIntro  {Γ φ ψ} (p₁ : LaxND Γ φ)
      (p₂ : LaxND Γ ψ) : LaxND Γ (.and φ ψ)
  | andElim1  {Γ φ ψ} (p : LaxND Γ (.and φ ψ)) :
      LaxND Γ φ
  | andElim2  {Γ φ ψ} (p : LaxND Γ (.and φ ψ)) :
      LaxND Γ ψ
  | orIntro1  {Γ φ ψ} (p : LaxND Γ φ) :
      LaxND Γ (.or φ ψ)
  | orIntro2  {Γ φ ψ} (p : LaxND Γ ψ) :
      LaxND Γ (.or φ ψ)
  | orElim    {Γ φ ψ χ} (p₀ : LaxND Γ (.or φ ψ))
      (p₁ : LaxND (φ :: Γ) χ)
      (p₂ : LaxND (ψ :: Γ) χ) : LaxND Γ χ
  | laxIntro  {Γ φ}   (p : LaxND Γ φ) :
      LaxND Γ (.somehow φ)
  | laxElim   {Γ φ ψ} (p₁ : LaxND Γ (.somehow φ))
      (p₂ : LaxND (φ :: Γ) (.somehow ψ)) :
      LaxND Γ (.somehow ψ)
```

The two lax rules are the parent paper's $\bigcirc$I/$\bigcirc$E; provability of φ is `Nonempty (LaxND [] φ)`. The IPL fragment is its own judgement (`IPLND`: the same rules minus the two lax ones),

and conservativity holds in erasure and classic forms (audits `[propext, Quot.sound]`):

```
theorem conservativity_prop {Γ φ} (p : LaxND Γ φ) :
    IPLND (Γ.map erase) (erase φ)

theorem conservativity_IPL {Γ φ} (hφ : isIPL φ)
    (hΓ : ∀ ψ ∈ Γ, isIPL ψ) (p : LaxND Γ φ) :
    IPLND Γ φ
```

A.2 Sequent calculus, cut elimination, Hilbert system

The G3-style, height-indexed, cut-free calculus (principal formulas kept in the context):

```
inductive SCh : Nat → List PLLFormula → PLLFormula →
    Prop
  | init  {n Γ a}     (h : PLLFormula.prop a ∈ Γ) :
      SCh n Γ (.prop a)
  | botL  {n Γ C}     (h : PLLFormula.falsePLL ∈ Γ) :
      SCh n Γ C
  | andR  {n Γ A B}   : SCh n Γ A → SCh n Γ B →
      SCh (n+1) Γ (A.and B)
  | andL  {n Γ A B C} (h : A.and B ∈ Γ) :
      SCh n (A :: B :: Γ) C → SCh (n+1) Γ C
  | orR1  {n Γ A B}   : SCh n Γ A →
      SCh (n+1) Γ (A.or B)
  | orR2  {n Γ A B}   : SCh n Γ B →
      SCh (n+1) Γ (A.or B)
  | orL   {n Γ A B C} (h : A.or B ∈ Γ) :
      SCh n (A :: Γ) C → SCh n (B :: Γ) C →
          SCh (n+1) Γ C
  | impR  {n Γ A B}   : SCh n (A :: Γ) B →
      SCh (n+1) Γ (A.ifThen B)
  | impL  {n Γ A B C} (h : A.ifThen B ∈ Γ) :
      SCh n Γ A → SCh n (B :: Γ) C → SCh (n+1) Γ C
  | laxR  {n Γ A}     : SCh n Γ A →
      SCh (n+1) Γ (A.somehow)
  | laxL  {n Γ A B}   (h : A.somehow ∈ Γ) :
      SCh n (A :: Γ) (B.somehow) →
          SCh (n+1) Γ (B.somehow)

def SC (Γ : List PLLFormula) (C : PLLFormula) :
    Prop :=
  ∃ n, SCh n Γ C
theorem SC.cut {Γ A C}
    (h₁ : SC Γ A) (h₂ : SC (A :: Γ) C) : SC Γ C

theorem cutElimination {Γ C} :
    Nonempty (LaxND Γ C) ↔ SC Γ C

theorem disjunction_property {A B}
    (h : Nonempty (LaxND [] (A ∨ B))) :
    Nonempty (LaxND [] A) ∨ Nonempty (LaxND [] B)

theorem somehow_reflection {A}
    (h : Nonempty (LaxND [] (○A))) :
    Nonempty (LaxND [] A)
```

All audit `[propext, Quot.sound]`. The Hilbert system has the three ○-schemes of Section 1 over the standard IPC schemes, with `hd_iff_ND` (audit `[propext]`) the equivalence with natural deduction.

A.3 Constraint semantics

```
structure ConstraintModel where
  W : Type
  Ri : W → W → Prop  -- intuitionistic accessibility
  Rm : W → W → Prop        -- constraint accessibility
  F : Set W                -- fallible worlds
  V : String → Set W
  refl_i : ∀ w, Ri w w
  trans_i : ∀ {w v u}, Ri w v → Ri v u → Ri w u
  refl_m : ∀ w, Rm w w
  trans_m : ∀ {w v u}, Rm w v → Rm v u → Rm w u
  sub_mi : ∀ {w v}, Rm w v → Ri w v
  hered_F : ∀ {w v}, Ri w v → w ∈ F → v ∈ F
  hered_V : ∀ {a w v}, Ri w v → w ∈ V a → v ∈ V a
  full_F : ∀ {a w}, w ∈ F → w ∈ V a

def force (C : ConstraintModel) : C.W → PLLFormula →
    Prop
  | w, .prop a     => w ∈ C.V a
  | w, .falsePLL   => w ∈ C.F
  | w, .and φ ψ    => C.force w φ ∧ C.force w ψ
  | w, .or φ ψ     => C.force w φ ∨ C.force w ψ
  | w, .ifThen φ ψ =>
      ∀ v, C.Ri w v → C.force v φ → C.force v ψ
  | w, .somehow φ  =>
      ∀ v, C.Ri w v → ∃ u, C.Rm v u ∧ C.force u φ

theorem soundness    {Γ φ} (p : LaxND Γ φ) : Γ ⊨- φ
theorem completeness {Γ φ} (h : Γ ⊨- φ) :
    Nonempty (LaxND Γ φ)
```

In the clauses of `force` the formula connectives occur only in the patterns on the left, as constructors. The $\wedge$, $\vee$, $\to$, $\forall$ and $\exists$ on the right are Lean's own, on propositions: the definition translates each formula connective into the corresponding statement about worlds.

`soundness` audits `[propext]`; `completeness` is the parent's Zorn construction, clean-classical and kept as the foil for Section 6.2. The three countermodels of the parent's Fig. 3 are explicit finite models, `decide`-checkable: `not_provable_not_somehow_false`, `not_provable_somehow_or_dist`, `not_provable_imp_somehow_dist`.

A.4 Decidability, and the repaired calculus

The terminating calculus `G4c` (with height-indexed form `G4h`) is the repaired `G4iLL″`; the published calculus is provably incomplete for PLL:

```
def Fa : PLLFormula :=
  ((prop "p").somehow).ifThen (prop "r")
def Ga : PLLFormula := Fa.ifThen (prop "p").somehow

theorem sep_SC :                    -- PLL derives it
  SC [○Ga, Fa] (prop "r")
theorem sep_not_G4 :                -- G4iLL cannot
  ¬ G4 [○Ga, Fa] (prop "r")
```

(Neither contraction nor cut is admissible in the published system: `contraction_not_admissible`, `cut_not_admissible`.) The repaired calculus proves exactly the PLL sequents, and a loop-checked, fuel-bounded backward search decides it — all `[propext, Quot.sound]`, no choice, and kernel-honest (no `native_decide` anywhere in the chain):

```
theorem G4c.equiv_tm {Γ φ} : G4c Γ φ ↔
    Nonempty (Tm Γ φ)

instance decidableG4c (Γ C) : Decidable (G4c Γ C)
instance decidablePLL (Γ φ) :
    Decidable (Nonempty (Tm Γ φ))
```

A.5 Craig interpolation

By a Maehara induction over `SCh`; the working atom bookkeeping is built on a choice-free finite-set toolkit, so the primary statements audit `[propext, Quot.sound]`:

```
theorem craig_interpolation' {Γ₁ Γ₂ C}
    (h : SC (Γ₁ ++ Γ₂) C) :
    ∃ I : PLLFormula,
      SC Γ₁ I ∧ SC (I :: Γ₂) C ∧
      I.atoms ⊆ atomsList Γ₁ ∧
      I.atoms ⊆ finUnion (atomsList Γ₂) C.atoms

theorem craig_implication' {A B}
    (h : SC [] (A.ifThen B)) :
    ∃ I : PLLFormula,
      SC [] (A.ifThen I) ∧ SC [] (I.ifThen B) ∧
      I.atoms ⊆ A.atoms ∧ I.atoms ⊆ B.atoms
```

The Mathlib-phrased forms, with the shared atoms stated through the lattice ∩ and ∪, are kept as two-line wrappers, clean‡ by statement vocabulary alone. The lax ○ contributes exactly one twist to the textbook induction: when the principal formula ○A sits in Γ_1, the premise interpolant is *boxed*, I ↦ ○I — forced because ○L fires only at a ○-shaped succedent.

B Evidence and the Clause Families

The equipment of Section 4, in full:

```
structure Pca where
  Carrier : Type
  app : Carrier → Carrier → Option Carrier
  pair : Carrier → Carrier → Carrier
  fst : Carrier → Carrier
  snd : Carrier → Carrier
  tag : Bool → Carrier → Carrier
  untag : Carrier → Bool × Carrier
  fst_pair : ∀ a b, fst (pair a b) = a
  snd_pair : ∀ a b, snd (pair a b) = b
  untag_tag : ∀ i a, untag (tag i a) = (i, a)

structure Evidence (P : Pca) (C : ConstraintModel)
    where
  E : String → C.W → Set P.Carrier
  hered_E : ∀ {s w v}, C.Ri w v → E s w ⊆ E s v
  full_E : ∀ {s w}, w ∈ C.F → ∀ x, x ∈ E s w
```

Uniform realisability, in full (`realU`; every world-quantified clause carries the fallibility guard):

```
def realU (P : Pca) {C} (Ev : Evidence P C) :
    PLLFormula → P.Carrier → C.W → Prop
  | .prop s, x, w => w ∈ C.F ∨ x ∈ Ev.E s w
  | .falsePLL, _x, w => w ∈ C.F
  | .and φ ψ, x, w =>
      w ∈ C.F ∨ (realU P Ev φ (P.fst x) w ∧
                 realU P Ev ψ (P.snd x) w)
  | .or φ ψ, x, w =>
      w ∈ C.F ∨
        ((P.untag x).1 = false ∧
          realU P Ev φ (P.untag x).2 w)
      ∨ ((P.untag x).1 = true ∧
          realU P Ev ψ (P.untag x).2 w)
  | .ifThen φ ψ, x, w =>
      ∀ v, C.Ri w v → v ∈ C.F ∨
        (∀ b, realU P Ev φ b v →
          ∃ y, P.app x b = some y ∧ realU P Ev ψ y v)
  | .somehow φ, x, w =>
      ∀ v, C.Ri w v → v ∈ C.F ∨
        (∃ u, C.Rm v u ∧ realU P Ev φ x u)
```

Strategy realisability `realS` differs at ○ alone (the clause displayed in Section 4): the realiser is applied to the code κ v of the presented future and must *name* its constraint-witness. Presented-strategy realisability `realP` differs from `realS` at ↠ alone (also displayed in Section 4): the realiser receives the pair of the presented world-code and the argument. Heredity along R_i and fallible saturation are theorems for all three relations (`realU_hered`, `realS_hered`, `realU_of_fallible`, `realS_of_fallible` — axiom-free — and their `realP` analogues).

The separations of Theorem 5.1, as mechanised:

```
theorem bite_uniform_split :
    modelOrSplit.force W3.r
      (○(prop "A" ∨ prop "B")) ∧
    ∀ x : ℕ,
      ¬ realU natPca
          (fullEvidence natPca modelOrSplit)
```

```
          (○(prop "A" ∨ prop "B")) x W3.r

theorem uniform_dist_valid
    (P : Pca) {C} (Ev : Evidence P C) (I : P.Carrier)
    (hI : ∀ b, P.app I b = some b)
    (φ ψ : PLLFormula) (w : C.W) :
    realU P Ev
      (○(φ ∨ ψ) ↠ ○φ ∨ ○ψ) I w

theorem impdist_not_uniform (I : P.Carrier)
    (hI : ∀ b, P.app I b = some b) (e : P.Carrier) :
    ¬ realU P (EvChain P)
        ((○(prop "A") ↠ ○(prop "B")) ↠
          ○(prop "A" ↠ prop "B"))
        e W3.r
```

(`uniform_dist_valid` is axiom-free.) The strategy pair `strategy_realises_obAB` / `strategy_dist_refuted` is stated for any structure whose application satisfies three equations packaging the case-splitting strategy, and instantiated over the $\mathbb{N}$-pairing structure (`strategy_realises_obAB_split`, `strategy_dist_refuted_split`).

The obstruction frame, as checker data, and the obstruction (audit `[propext, Quot.sound]`):

```
abbrev obsM : FinCM :=
  { n := 3, ri := [(0, 1), (0, 2)], rm := [],
    fall := [],
    val := [(1, "p"), (2, "q"), (1, "t"), (2, "t")] }

theorem realS_fullness_obstruction
    (P : Pca) (κ : obsC.W → P.Carrier)
    (htab : ∀ g : obsC.W → P.Carrier,
      ∃ s, ∀ i, P.app s (κ i) = some (g i)) :
    ¬ ∃ Ev : Evidence P obsC,
      (∀ (a : String) (w : obsC.W),
        ∀ x ∈ Ev.E a w, w ∈ obsC.V a) ∧
      (∀ (φ : PLLFormula) (w : obsC.W),
        obsC.force w φ → ∃ x, x ⊩ˢ[Ev, κ, w] φ)
```

The extraction results cited in Section 7: combinatory completeness for structures with k/s (`Poly.abs_spec`, `[propext, Quot.sound]`), and soundness-with-extraction `extract_sound` (uniform) and `extractS_sound` (strategy): a natural-deduction derivation compiles to a polynomial whose value realises the conclusion wherever the context is realised.

C The Finite Canonical Model

Theories are triples; promises are guarded by ○:

```
structure FTheory where
  val  : Finset PLLFormula   -- validated (believed)
  fal  : Finset PLLFormula   -- falsified here
  mfal : Finset PLLFormula   -- promised: falsified at
                             --   every Rₘ-successor

-- Consistent ⟨V, F, M⟩ iff for no finite
-- Ds ⊆ F, Ts ⊆ M (jointly nonempty):
--   V ⊢ ⋁Ds ∨ ○(⋁Ts)
```

Consistency of a finite triple collapses to its worst case — one derivability question, decided by `decidablePLL` (`cons_iff_rep`, `[propext, Quot.sound]`; the `Finset.toList`-phrased form `cons_iff_check` is its clean‡ wrapper) — and the Lindenbaum construction is a decided fold:

```
noncomputable def extendStep
    (T : FTheory) (φ : PLLFormula) : FTheory :=
  if (T.insVal φ).Cons
  then T.insVal φ else T.insFal φ

theorem lindenbaum {cl : Finset PLLFormula}
    {T : FTheory}
    (hT : T.Cons) (hIn : InCl cl T) :
    ∃ T', T.le T' ∧ MaxIn cl T' ∧ T'.mfal = T.mfal
```

`MaxIn cl T` — totality over the closure — is the finite surrogate for maximality; the else-branch of the fold is safe by the extension dichotomy `cons_insVal_or_insFal`, and the fold never writes to the promises, an equality the truth lemma uses twice.

```
def canonFin (cl : Finset PLLFormula) :
    ConstraintModel where
  W  := {T : FTheory // MaxIn cl T}
  Ri T T' := T.1.val ⊆ T'.1.val
  Rm T T' := T.1.val ⊆ T'.1.val ∧ T.1.mfal ⊆ T'.1.mfal
  F  := {T | falsePLL ∈ T.1.val}
  V a := {T | prop a ∉ cl ∨ prop a ∈ T.1.val}

theorem truth_lemma {cl} (hcl : SubClosed cl) :
    ∀ (φ : PLLFormula), φ ∈ cl →
      ∀ T : (canonFin cl).W,
      (φ ∈ T.1.val → (canonFin cl).force T φ) ∧
        (φ ∈ T.1.fal → ¬ (canonFin cl).force T φ)

theorem emitter_completeness
    {Γ : List PLLFormula} {C : PLLFormula}
    (h : ¬ Nonempty (LaxND Γ C)) :
    ∃ (M : FinCM) (w : Nat),
      FinCM.checkB M w Γ C = true
```

All `[propext, Quot.sound]`. The three Lindenbaum extensions of the truth lemma are exactly the parent's, with the start-triples carried over unchanged (Section 6.2).

Finally, the two decoration witnesses elided from the proof of Theorem 6.1. At ∨, fullness computes the branch by running the executable forcing of the left disjunct:

```
fun w =>
  if M.forceB w.1 φ = true
  then P.tag false (witφ w)
  else P.tag true (witψ w)
```

If the test succeeds, the left inductive fullness supplies the payload; if it fails while the disjunction is forced, the failed

computation refutes the left disjunct outright, so the right one is forced. At ○, fullness tabulates a bounded search of the finite frame:

```
fun v => match (List.finRange M.n).find?
    (fun u => M.rmB v.1 u.1 && M.forceB u.1 φ) with
  | some u => P.pair (κ u) (witφ u)
  | none   => P.pair (κ v) (witφ v)  -- unreachable
                                     --   when obliged
```

When $\bigcirc\varphi$ is forced, the $\forall\exists$ clause guarantees the search succeeds; the default branch exists only to make the function total.

D The Term Calculus

The proof-term calculus `Tm` (intrinsically typed, the computational lambda-calculus over the natural deduction of Appendix A), its full one-step reduction `Step` (β for every connective together with the monadic `let`-associativity law), strong normalisation `SNt` and normal forms `Nf` are a full calculus, pointed to rather than reproduced. The audited statements:

```
theorem strong_normalisation {Γ φ} (t : Tm Γ φ) :
    SNt t

def Tm.normalize {Γ φ} (t : Tm Γ φ) : Tm Γ φ
theorem Tm.normalize_spec {Γ φ} (t : Tm Γ φ) :
    Steps t t.normalize ∧ Nf t.normalize
```

Strong normalisation is by Lindley–Stark ⊤⊤-lifting [46] — the β-fragment alone admits ordinary Tait reducibility, but the artefact also proves, by two machine-checked terms, that the β- and associativity fragments do not compose by any syntactic argument (each creates redexes of the other), so the semantic method is needed. Confluence is Newman's lemma [52] from strong normalisation (`local_confluence`, `confluence`), whence uniqueness of normal forms and decidability of conversion by the normaliser (`normal_form_unique`, `conv_iff_normalize_eq`); Proposition 7.2 is `mu_eta_not_mutually_inverse` (the complete four-term reduction graph of $\eta \circ \mu$, `[propext]` alone) together with `eta_mu_not_conv_id` (via confluence).

E Artefact Guide

The artefact is a Lean 4 (v4.31.0, Mathlib-based) library; `lake build` checks every result, including the pinned axiom audits — each named theorem carries `#print axioms` under `#guard_msgs`, so the build fails if any audit drifts. There are no `sorry`s in the library and no `native_decide` on any claimed result.

Orientation, by the sections of this paper:

result	Lean name	audit
cut elimination	`cutElimination`	[p,Q]
disjunction property	`disjunction_property`	[p,Q]
○-reflection	`somehow_reflection`	[p,Q]
conservativity over IPL	`conservativity_IPL`	[p,Q]
strong normalisation (β + assoc)	`strong_normalisation`	clean
decidability (parent Thm 2.8)	`decidablePLL`	[p,Q]
published G4iLL incomplete	`sep_not_G4`	[propext]
Craig interpolation	`craig_interpolation'`	[p,Q]
Kleene–Brouwer well-foundedness	`wellFounded_kb`	none
collapse, N(B) ≅ B	`nucleus_eq_sup_bot`, `nucleusOrderIsoBot`	clean
open = closed iff EM at a	`em_of_openNucleus_eq_...`	clean
infinite closed fragment	`closed_lax_infinite`	clean
context completeness	`Ctx.thm6`, `Ctx.corollary10`	clean
doxastic profile	`BeliefIdealisation/Normality/Falsum`	clean
3-chain enumeration	`chain3_card` (+ separation)	clean
the separations	`Belief/Realisability.lean`	see file
the obstruction	`realS_fullness_obstruction`	[p,Q]
adequacy + fullness	`realP_adequate_and_full`	[p,Q]
refutation lemma (closed form)	`realP_refutes_sequent_tbl`	[p,Q]
decided Lindenbaum, truth lemma	`lindenbaum`, `truth_lemma`	[p,Q]
emitter completeness	`emitter_completeness`	[p,Q]
the crown	`derivable_iff_no_realP_refutation`	[p,Q]
idempotence ≠ isomorphism	`mu_eta_not_mutually_inverse`	[propext]

A statement-level ledger (every named result, its location and its measured audit, with the definitions reproduced verbatim) and line-level annotated walk-throughs of the three central proofs accompany the artefact.

Index of declarations

Results and definitions cited in the text as [D n], in order of first citation: the Lean name, its file, and its audit, as recorded in the artefact's proof-status ledger. Each name is linked to its declaration in the public repository. File names are relative to `LaxLogic/` unless they begin `wip/`.

The audit column, here and in the orientation table above, gives the axioms a declaration depends on:

none	no axioms
[propext]	`propext` only
[p,Q]	`propext`, `Quot.sound`: no choice
clean	`propext`, `Classical.choice`, `Quot.sound`

[D1] `PLLFormula`, `PLL/Syntax/Formula.lean`; none.
[D2] `PLLND.LaxND`, `PLL/ND/NDCore.lean`; none.
[D3] `PLLND.Tm`, `PLL/ND/Terms.lean`; none.
[D4] `PLLND.SC`, `PLL/Sequent/Sequent.lean`; none.
[D5] `PLLND.conservativity_IPL`, `PLL/ND/NDCore.lean`; [p,Q].
[D6] `PLLND.craig_interpolation'`, `PLL/Sequent/Craig.lean`; [p,Q].
[D7] `PLLND.soundness`, `PLL/Semantics/Kripke.lean`; [propext].
[D8] `PLLND.completeness`, `PLL/Semantics/Completeness.lean`; clean.
[D9] `BeliefLax.nucleus_eq_sup_bot`, `Belief/Collapse.lean`; clean.
[D10] `BeliefLax.nucleusOrderIsoBot`, `Belief/BooleanIso.lean`; clean.
[D11] `BeliefLax.em_of_openNucleus_eq_closedNucleus`, `Belief/OpenClosed.lean`; [propext].
[D12] `PLLND.LaxInfinite.closed_lax_infinite`, `PLL/Semantics/LaxInfinite.lean`; clean.
[D13] `BeliefLax.chain3_card`, `Belief/Examples.lean`; clean.
[D14] `BeliefLax.chain3_open_ne_closed`, `Belief/Examples.lean`; clean.
[D15] `PLLND.Ctx.thm6`, `PLL/Semantics/CtxCompleteness.lean`; clean.
[D16] `PLLND.Ctx.corollary10`, `PLL/Semantics/CtxCompleteness.lean`; clean.
[D17] `PLLND.Ctx.thm2_boolean_algebra`, `PLL/Semantics/CtxCompleteness.lean`; [p,Q].
[D18] `PLLND.BeliefReal.bite_uniform_split`, `Belief/Realisability.lean`; clean.

[D19] PLLND.BeliefReal.uniform_dist_valid, Belief/Realisability.lean; none.
[D20] PLLND.not_provable_somehow_or_dist, PLL/Semantics/Frames.lean; clean.
[D21] PLLND.BeliefReal.strategy_realises_obAB, Belief/Realisability.lean; clean.
[D22] PLLND.BeliefReal.strategy_dist_refuted, Belief/Realisability.lean; clean.
[D23] PLLND.BeliefReal.impdist_not_uniform, Belief/Realisability.lean; clean.
[D24] PLLND.BeliefReal.realS_fullness_obstruction, Belief/Realisability.lean; [p,Q].
[D25] PLLND.FinCM, PLL/Semantics/CountermodelEmit.lean; none.
[D26] PLLND.FinCM.forceB, PLL/Semantics/CountermodelEmit.lean; [p,Q].
[D27] PLLND.FinCM.force_iff, PLL/Semantics/CountermodelEmit.lean; [p,Q].
[D28] PLLND.BeliefReal.realP_adequate_and_full, PLL/Realisability/Evidence.lean; [p,Q].
[D29] PLLND.BeliefReal.realP_refutes_sequent, PLL/Realisability/Evidence.lean; [p,Q].
[D30] PLLND.BeliefReal.Tbl, PLL/Realisability/Evidence.lean; none.
[D31] PLLND.BeliefReal.tblPca, PLL/Realisability/Evidence.lean; [propext].
[D32] PLLND.BeliefReal.realP_refutes_sequent_tbl, PLL/Realisability/Evidence.lean; [p,Q].
[D33] PLLND.FinComp.FTheory, PLL/Semantics/FinComp.lean; [p,Q].
[D34] PLLND.FinComp.cons_iff_check, PLL/Semantics/FinComp.lean; clean.
[D35] PLLND.FinComp.cons_insVal_or_insFal, PLL/Semantics/FinComp.lean; [p,Q].
[D36] PLLND.FinComp.lindenbaum, PLL/Semantics/FinComp.lean; [p,Q].
[D37] PLLND.FinComp.truth_lemma, PLL/Semantics/FinComp.lean; [p,Q].
[D38] PLLND.FinComp.finite_canonical_countermodel, PLL/Semantics/FinComp.lean; [p,Q].
[D39] PLLND.FinComp.emitter_completeness, PLL/Semantics/FinComp.lean; [p,Q].
[D40] PLLND.BeliefReal.derivable_iff_no_realP_refutation, PLL/Realisability/RealCompleteness.lean; [p,Q].
[D41] PLLND.CounterEmit.emit, PLL/Semantics/CountermodelEmit.lean; [p,Q].
[D42] PLLND.demoM, PLL/Semantics/CountermodelEmit.lean; none.
[D43] PLLND.CounterEmit.emitMin, PLL/Semantics/CountermodelEmit.lean; [p,Q].
[D44] PLLND.CounterEmit.emitMinClean, PLL/Semantics/CountermodelEmit.lean; [p,Q].
[D45] PLLND.ConstraintModel.force_hered, PLL/Semantics/Kripke.lean; none.
[D46] PLLND.Diagram.riEquiv, PLL/Search/Diagram.lean; [p,Q].
[D47] PLLND.Diagram.rmEquiv, PLL/Search/Diagram.lean; [p,Q].
[D48] PLLND.BeliefReal.extract_sound, Belief/Realisability.lean; clean.
[D49] PLLND.BeliefReal.ob_strength, Belief/Realisability.lean; none.
[D50] PLLND.mu_eta_not_mutually_inverse, PLL/ND/Idempotency.lean; [propext].
[D51] PLLND.eta_mu_not_conv_id, PLL/ND/Idempotency.lean; clean.
[D52] PLLND.falsePath_beats_topological, PLL/Timing/Timing.lean; none.
[D53] PLLND.skip_beats_topological, PLL/Timing/TimingAdder.lean; none.
[D54] PLLND.RippleN.ready_bound, PLL/Timing/TimingRipple.lean; [p,Q].
[D55] PLLND.RippleN.ripple_tight, PLL/Timing/TimingRipple.lean; [p,Q].
[D56] PLLND.Lookahead.lookahead_beats_ripple, PLL/Timing/TimingLookahead.lean; clean.
[D57] PLLND.Lookahead.balGP_eq_linGP, PLL/Timing/TimingLookahead.lean; clean.
[D58] PLLND.Lookahead.lookahead_eq_ripple_gp, PLL/Timing/TimingLookahead.lean; clean.
[D59] PLLND.Async.cElem_beats_sequential, PLL/Timing/Async.lean; clean.
[D60] PLLND.Async.arbiter_no_distribution, PLL/Timing/Async.lean; [p,Q].
[D61] PLLND.Async.arbiter_not_confluent, PLL/Timing/Async.lean; [p,Q].
[D62] PLLND.ConfluentU.derivU_iff_confluent_valid, PLL/Semantics/ConfluentComplete.lean; clean.
[D63] PLLND.BeliefReal.force_somehow_iff_notnot, Belief/Realisability.lean; clean.
[D64] PLLND.force_somehow_iff_of_confluent, PLL/Semantics/Frames.lean; none.
[D65] PLLND.force_somehow_or_dist_of_confluent, PLL/Semantics/Frames.lean; [propext].
[D66] PLLND.BeliefReal.modelOrSplit_not_confluent, Belief/Realisability.lean; clean.

References

[1] Martín Abadi. 2006. Access Control in a Core Calculus of Dependency. In *Proceedings of the 11th ACM SIGPLAN International Conference on Functional Programming (ICFP 2006)*. ACM. The "says" modality of authorization logic as a monadic (lax-style) modality indexed by a principal: a published non-factive, agent-flavoured reading of the computational lambda-calculus.

[2] Andreas Abel, Guillaume Allais, Aliya Hameer, Alberto Momigliano, Brigitte Pientka, Steven Schäfer, and Kathrin Stark. 2019. POPLMark Reloaded: Mechanizing Proofs by Logical Relations. *Journal of Functional Programming* 29 (2019). Formal development: strong normalisation by logical relations mechanised in Beluga, Coq and Agda; the benchmark neighbour of the artefact's $\top\top$-lifting mechanisation.

[3] Sergei Artemov. 2001. Explicit Provability and Constructive Semantics. *The Bulletin of Symbolic Logic* 7, 1 (2001), 1–36.

[4] Sergei Artemov. 2008. The Logic of Justification. *The Review of Symbolic Logic* 1, 4 (2008), 477–513.

[5] Sergei Artemov and Melvin Fitting. 2019. *Justification Logic: Reasoning with Reasons*. Number 216 in Cambridge Tracts in Mathematics. Cambridge University Press.

[6] Sergei Artemov and Tudor Protopopescu. 2016. Intuitionistic Epistemic Logic. *The Review of Symbolic Logic* 9, 2 (2016), 266–298. arXiv:1406.1582 IEL and the belief system IEL$^-$ (co-reflection without factivity); the nearest published neighbour of the doxastic reading of the lax modality.

[7] Francisco Ávila, Guram Bezhanishvili, Patrick J. Morandi, and Angel Zaldívar. 2020. When Is the Frame of Nuclei Spatial: A New Approach. *Journal of Pure and Applied Algebra* 224, 7 (2020), 106302. doi:10.1016/j.jpaa.2019.106302 arXiv:1906.03636 Nuclei via Esakia-dual nuclear sets; recovers Beazer–Macnab, Simmons, Niefield–Rosenthal and Isbell as corollaries.

[8] Alexandru Baltag, Nick Bezhanishvili, Aybüke Özgün, and Sonja Smets. 2016. Justified Belief and the Topology of Evidence. In *Logic, Language, Information, and Computation (WoLLIC 2016) (Lecture Notes in Computer Science, Vol. 9803)*. Springer, 83–103. doi:10.1007/978-3-662-52921-8_6

[9] Alexandru Baltag, Nick Bezhanishvili, Aybüke Özgün, and Sonja Smets. 2022. Justified Belief, Knowledge, and the Topology of Evidence. *Synthese* 200, 6 (2022). Journal version of BaltagBezhanishviliOzgunSmets2016.

[10] Rod Beazer and Donald S. Macnab. 1979. Modal Extensions of Heyting Algebras. *Colloquium Mathematicum* 41, 1 (1979), 1–12. If L is Boolean then $N(L) \cong L$: the classical collapse of the lattice of belief modalities. Formal development: the collapse (every nucleus on a Boolean algebra is closed, incomplete case included) is machine-checked in Lean 4 in the present artefact (`Belief/Collapse.lean`, `Belief/BooleanIso.lean`).

[11] P. N. Benton, G. M. Bierman, and V. C. V. de Paiva. 1998. Computational Types from a Logical Perspective. *Journal of Functional Programming* 8, 2 (1998), 177–193.

[12] Kevin Buzzard, Johan Commelin, and Patrick Massot. 2020. Formalising Perfectoid Spaces. In *Certified Programs and Proofs (CPP 2020)*. ACM. Formal development: Lean. Precedent for citing a Lean repository as the artefact of a refereed paper (see docs/publication-routes.md).

[13] Liron Cohen, Ariel Grunfeld, Dominik Kirst, and Étienne Miquey. 2025. Syntactic Effectful Realizability in Higher-Order Logic. In *40th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS 2025)*. arXiv:2506.09458 EffHOL: realizability translation parametrised by a monad; every instance induces an evidenced frame, tripos and realizability topos.

[14] Haskell B. Curry. 1950. *A Theory of Formal Deducibility*. Number 6 in Notre Dame Mathematical Lectures. University of Notre Dame Press, Notre Dame, Indiana. Based on lectures delivered at Notre Dame in 1948; second edition 1957.

[15] Haskell B. Curry. 1952. The Elimination Theorem When Modality Is Present. *The Journal of Symbolic Logic* 17, 4 (1952), 249–265. Cut elimination in the presence of the possibility-like modality of the 1948 lectures; the historical source of the lax modality.

[16] Leonardo de Moura and Sebastian Ullrich. 2021. The Lean 4 Theorem Prover and Programming Language. In *Automated Deduction – CADE 28 (Lecture Notes in Computer Science, Vol. 12699)*. Springer, 625–635. Formal development: the proof assistant of the artefact.

[17] Christian Doczkal and Jan Schwinghammer. 2009. Formalizing a Strong Normalization Proof for Moggi's Computational Metalanguage: A Case Study in Isabelle/HOL-Nominal. In *Logical Frameworks and Meta-Languages: Theory and Practice (LFMTP 2009)*. ACM. Formal development: the sole prior mechanisation of ⊤⊤-lifting, for the core calculus without sums; maintained as Isabelle AFP entry `Lam-ml-Normalization` (2010).

[18] Roy Dyckhoff. 1992. Contraction-free Sequent Calculi for Intuitionistic Logic. *The Journal of Symbolic Logic* 57, 3 (1992), 795–807. G4ip, the template for terminating contraction-free calculi. Formal development: implemented as Mathlib's `itauto` (termination trusted, output kernel-checked); mechanised termination for G4-style calculi appears in the Coq iSL line (ShillitoVanDerGiessenGoreIemhoff2023).

[19] Marcel Erné. 2022. Nuclear Ranges in Implicative Semilattices. *Algebra Universalis* 83, 2 (2022), article 18. doi:10.1007/s00012-022-00768-3 The decisive source on nuclei over NON-complete Heyting algebras: the nuclei form a bounded meet-semilattice, and existence of joins and implication of nuclei is completeness-gated (necessary and sufficient conditions).

[20] Marcel Erné, Jorge Picado, and Aleš Pultr. 2022. Adjoint Maps Between Implicative Semilattices and Continuity of Localic Maps. *Algebra Universalis* 83, 2 (2022), article 19. doi:10.1007/s00012-022-00767-4

[21] Ronald Fagin, Joseph Y. Halpern, Yoram Moses, and Moshe Y. Vardi. 1995. *Reasoning About Knowledge*. MIT Press, Cambridge, MA. Standard reference for classical doxastic logic (KD45) and logical omniscience.

[22] Matthew Fairtlough. 2026. lax-logic-in-lean: A Lean 4 Formalisation of Propositional Lax Logic and the Belief Semantics. https://github.com/fairflow/lax-logic-in-lean. doi:10.5281/zenodo.XXXXXXX PLACEHOLDER DOI – mint via Zenodo release of the tagged artefact commit and fill in; add `CITATION.cff`. Formal development: Lean 4 + Mathlib; axiom audit per theorem recorded in docs/pll-formalisation-ledger.md.

[23] Matt Fairtlough and Michael Mendler. 1995. An Intuitionistic Modal Logic with Applications to the Formal Verification of Hardware. In *Computer Science Logic (CSL 1994) (Lecture Notes in Computer Science, Vol. 933)*. Springer. doi:10.1007/BFb0022268

[24] Matt Fairtlough and Michael Mendler. 1997. Propositional Lax Logic. *Information and Computation* 137, 1 (1997), 1–33. doi:10.1006/inco.1997.2627 Formal development: the entire paper (calculi, equivalences, cut elimination, constraint semantics, soundness, completeness, decidability) is mechanised in Lean 4 in the present artefact (`lax-logic-in-lean`).

[25] Matt Fairtlough and Michael Mendler. 2002. On the Logical Content of Computational Type Theory: A Solution to Curry's Problem. In *Types for Proofs and Programs (TYPES 2000) (Lecture Notes in Computer Science, Vol. 2277)*, Paul Callaghan, Zhaohui Luo, James McKinna, and Robert Pollack (Eds.). Springer, 63–78. doi:10.1007/3-540-45842-5_5 Formal development: the context-completeness theorem (Thm 6) and the infinitude of the closed lax fragment are mechanised in Lean 4 in the present artefact (`PLL/Semantics/CtxCompleteness.lean`, `PLL/Semantics/LaxInfinite.lean`).

[26] Matt Fairtlough, Michael Mendler, and Michael Walton. 1997. *First-order Lax Logic as a Framework for Constraint Logic Programming*. Technical Report MIP-9714. University of Passau.

[27] Hugo Férée, Iris van der Giessen, Sam van Gool, and Ian Shillito. 2024. Mechanised Uniform Interpolation for Modal Logics K, GL, and iSL. In *Automated Reasoning (IJCAR 2024) (Lecture Notes in Computer Science, Vol. 14740)*. Springer, 43–61. doi:10.1007/978-3-031-63501-4_3 arXiv:2402.10494 Best Paper Award. Formal development: uniform interpolants mechanised in Coq (`hferee/UIML`), with extracted interpolant calculators; the reference point for the open mechanised-UI question for PLL.

[28] Melvin Fitting. 2005. The Logic of Proofs, Semantically. *Annals of Pure and Applied Logic* 132, 1 (2005), 1–25. Evidence-function possible-world models; the monotone evidence condition mirrored by the artefact's hereditary Evidence structure.

[29] Deepak Garg and Martín Abadi. 2008. A Modal Deconstruction of Access Control Logics. In *Foundations of Software Science and Computational Structures (FoSSaCS 2008) (Lecture Notes in Computer Science, Vol. 4962)*. Springer. doi:10.1007/978-3-540-78499-9_16

[30] Malvin Gattinger. 2026. lean4-pdl: Towards Machine-Checked Interpolation for PDL in Lean 4. https://github.com/m4lvin/lean4-pdl. Formal development: Lean 4; the one live Lean interpolation project, neighbour to the artefact's open PLL uniform-interpolation task.

[31] Gerhard Gentzen. 1935. Untersuchungen über das logische Schließen. I, II. *Mathematische Zeitschrift* 39 (1935), 176–210, 405–431. English translation in M. E. Szabo (ed.), The Collected Papers of Gerhard Gentzen, North-Holland, 1969. Decidability of intuitionistic propositional logic from a terminating calculus, thirty years before its Kripke completeness. Formal development: cut elimination is mechanised in many systems, incl. Lean 4 (FormalizedFormalLogic/Foundation) and the present artefact's cut-free sequent calculus for PLL (`PLL/Sequent/Sequent.lean`).

[32] Robert I. Goldblatt. 1981. Grothendieck Topology as Geometric Modality. *Zeitschrift für mathematische Logik und Grundlagen der Mathematik* 27 (1981), 495–529. doi:10.1002/malq.19810273104

[33] Christian Hagemeier and Dominik Kirst. 2022. Constructive and Mechanised Meta-Theory of Intuitionistic Epistemic Logic. In *Logical Foundations of Computer Science (LFCS 2022) (Lecture Notes in Computer Science, Vol. 13137)*. Springer. doi:10.1007/978-3-030-93100-1_7 Formal development: constructive completeness, semantic cut-elimination and FMP for Artemov–Protopopescu's IEL, mechanised in Coq (`uds-psl/intuitionistic-epistemic-logic`); journal version in Journal of Logic and Computation 32(8), 2022. The nearest existing mechanised-epistemic-logic artefact to the present development.

[34] Jaakko Hintikka. 1962. *Knowledge and Belief: An Introduction to the Logic of the Two Notions*. Cornell University Press, Ithaca, NY.

[35] Jacob M. Howe. 2001. Proof Search in Lax Logic. *Mathematical Structures in Computer Science* 11, 4 (2001), 573–588. doi:10.1017/S0960129501003334

[36] J. Martin E. Hyland. 1982. The Effective Topos. In *The L.E.J. Brouwer Centenary Symposium*, Anne S. Troelstra and Dirk van Dalen (Eds.). North-Holland, Amsterdam, 165–216.

[37] Rosalie Iemhoff. 2022. The G4i Analogue of a G3i Sequent Calculus. *Studia Logica* 110 (2022), 1493–1506. doi:10.1007/s11225-022-10008-3 arXiv:2011.11847 The general G3i-to-G4i transfer theorem invoked by Iemhoff2024; its modal case fails on the lax instance (machine-checked in the present artefact).

[38] Rosalie Iemhoff. 2024. Proof Theory for Lax Logic. In *Dick de Jongh on Intuitionistic and Provability Logics*, Nick Bezhanishvili, Rosalie Iemhoff, and Fan Yang (Eds.). Outstanding Contributions to Logic, Vol. 28. Springer, 203–229. doi:10.1007/978-3-031-47921-2_8 arXiv:2209.08976 Introduces the terminating calculus G4iLL and claims uniform interpolation for PLL. Formal development: the present artefact contains a machine-checked counterexample to the calculus's completeness (and to cut/contraction admissibility) together with a repaired calculus (`PLLG4Gap.lean`), communicated to the author; the uniform-interpolation claim is therefore open as published.

[39] Peter T. Johnstone. 1982. *Stone Spaces*. Cambridge Studies in Advanced Mathematics, Vol. 3. Cambridge University Press. Chapter II.2: nuclei and the assembly; joins of nuclei are not pointwise.

[40] Alexander S. Kechris. 1995. *Classical Descriptive Set Theory*. Graduate Texts in Mathematics, Vol. 156. Springer. doi:10.1007/978-1-4612-4190-4 Standard reference for the Kleene–Brouwer (Lusin–Sierpiński) order. Formal development: well-foundedness of the KB order on an inductively well-founded tree is mechanised, fully constructively (empty axiom footprint), in the present artefact (`Util/KleeneBrouwer.lean`).

[41] Stephen C. Kleene. 1945. On the Interpretation of Intuitionistic Number Theory. *The Journal of Symbolic Logic* 10, 4 (1945), 109–124.

[42] Satoshi Kobayashi. 1997. Monad as Modality. *Theoretical Computer Science* 175, 1 (1997), 29–74.

[43] Saul A. Kripke. 1965. Semantical Analysis of Intuitionistic Logic I. In *Formal Systems and Recursive Functions*, John N. Crossley and Michael A. E. Dummett (Eds.). North-Holland, Amsterdam, 92–130. Formal development: Kripke semantics for intuitionistic and modal logics has multiple mechanisations; the F&M constraint refinement is mechanised in the present artefact (`PLL/Semantics/Kripke.lean`).

[44] Jean-Louis Krivine. 2009. Realizability in Classical Logic. In *Interactive Models of Computation and Program Behaviour*. Panoramas et Synthèses, Vol. 27. Société Mathématique de France. The far-classical end of "the evidence discipline determines the logic": control operators realise classical principles.

[45] Sam Lindley. 2005. *Normalisation by Evaluation in the Compilation of Typed Functional Programming Languages*. Ph. D. Dissertation. University of Edinburgh. Section 3.5: strong normalisation for λ_{ml} with unrestricted sums via frame stacks, the paper-form counterpart of the sums case of `PLL/Normalisation/TopTop.lean`; the thesis also proves confluence and decidable convertibility for λ_{ml}, cf. `PLL/Normalisation/Confluence.lean`.

[46] Sam Lindley and Ian Stark. 2005. Reducibility and $\top\top$-Lifting for Computation Types. In *Typed Lambda Calculi and Applications (TLCA 2005) (Lecture Notes in Computer Science, Vol. 3461)*. Springer, 262–277. doi:10.1007/11417170_20 Formal development: the $\top\top$-lifting strong-normalisation argument is mechanised in Lean 4 in the present artefact (`PLL/Normalisation/TopTop.lean`), apparently the first mechanised $\top\top$-lifting with sums.

[47] James Lipton. 1992. Constructive Kripke Semantics and Realizability. In *Logic from Computer Science*, Yiannis N. Moschovakis (Ed.). Mathematical Sciences Research Institute Publications, Vol. 21. Springer. UNVERIFIED: page range not confirmed; content known from secondary citations only (Kripke nodes each carrying realizability structure).

[48] Donald S. Macnab. 1981. Modal Operators on Heyting Algebras. *Algebra Universalis* 12, 1 (1981), 5–29. doi:10.1007/BF02483860 Nuclei (= interpretations of the lax modality) on possibly non-complete Heyting algebras.

[49] Sonia Marin, Meghna Padhiar, and Ian Shillito. 2026. Intuitionistic Justification Logic, Semantically. arXiv:2606.31884 Intuitionistic-base modular models fusing an evidence function with Kripke machinery; the right base logic for a justification-style reading of the lax modality.

[50] Michael Mendler. 2000. Characterising Combinational Timing Analyses in Intuitionistic Modal Logic. *Logic Journal of the IGPL* 8, 6 (2000), 821–852. The timing-analysis perspective on the lax modality (sixth perspective of FairtloughMendler1997).

[51] Eugenio Moggi. 1991. Notions of Computation and Monads. *Information and Computation* 93, 1 (1991), 55–92. doi:10.1016/0890-5401(91)90052-4 Formal development: the computational lambda-calculus is mechanised as an intrinsically-typed term calculus in the present artefact (`PLL/ND/Terms.lean`).

[52] Maxwell H. A. Newman. 1942. On Theories with a Combinatorial Definition of "Equivalence". *Annals of Mathematics* 43, 2 (1942), 223–243. Newman's lemma. Formal development: mechanised in Lean 4 in the present artefact (`PLL/Normalisation/Confluence.lean`: confluence of the lax term calculus from strong normalisation); also available in Mathlib and in standard Coq/Isabelle rewriting libraries.

[53] Iwao Nishimura. 1960. On Formulas of One Variable in Intuitionistic Propositional Calculus. *The Journal of Symbolic Logic* 25 (1960), 327–331. The Rieger–Nishimura lattice: the free Heyting algebra on one generator is infinite.

[54] Frank Pfenning and Rowan Davies. 2001. A Judgmental Reconstruction of Modal Logic. *Mathematical Structures in Computer Science* 11, 4 (2001), 511–540. doi:10.1017/S0960129501003322 Decomposes the lax modality as possibility-following-necessity. Formal development: Fitch-style/judgmental modal calculi have Agda mechanisations, e.g. `nachivpn/k` (see ValliappanRuchTomeCortinas2022).

[55] Jorge Picado and Aleš Pultr. 2012. *Frames and Locales: Topology Without Points*. Birkhäuser. doi:10.1007/978-3-0348-0154-6

[56] Andrew M. Pitts. 1992. On an Interpretation of Second Order Quantification in First Order Intuitionistic Propositional Logic. *The Journal of Symbolic Logic* 57, 1 (1992), 33–52. The origin of proof-theoretic uniform interpolation, the method behind the G4-style route attempted for PLL.

[57] Ladislav Rieger. 1949. On the Lattice Theory of Brouwerian Propositional Logic. *Acta Facultatis Rerum Naturalium Universitatis Carolinae* 189 (1949). UNVERIFIED: canonical citation; primary source not reached.

[58] Ian Shillito, Iris van der Giessen, Rajeev Goré, and Rosalie Iemhoff. 2023. A New Calculus for Intuitionistic Strong Löb Logic: Strong Termination and Cut-Elimination, Formalised. In *Automated Reasoning with Analytic Tableaux and Related Methods (TABLEAUX 2023) (Lecture Notes in Computer Science, Vol. 14278)*. Springer. doi:10.1007/978-3-031-43513-3_5 arXiv:2309.00486 Formal development: all proofs formalised in Coq; the model for verified terminating G4-style calculi that the artefact's repaired PLL calculus joins.

[59] Alex K. Simpson. 1994. *The Proof Theory and Semantics of Intuitionistic Modal Logic*. Ph. D. Dissertation. University of Edinburgh. Birelational (intuitionistic-plus-modal accessibility) semantics, the frame format shared by the F&M constraint models.

[60] Robert Stalnaker. 2006. On Logics of Knowledge and Belief. *Philosophical Studies* 128, 1 (2006), 169–199.

[61] The mathlib Community. 2020. The Lean Mathematical Library. In *Certified Programs and Proofs (CPP 2020)*. ACM, 367–381. doi:10.1145/3372885.3373824 Formal development: Mathlib, the library the artefact builds on; includes `Order/Nucleus.lean` (nuclei on frames).

[62] Anne S. Troelstra (Ed.). 1973. *Metamathematical Investigation of Intuitionistic Arithmetic and Analysis*. Lecture Notes in Mathematics, Vol. 344. Springer. doi:10.1007/BFb0066739

[63] Anne S. Troelstra. 1998. Realizability. In *Handbook of Proof Theory*, Samuel R. Buss (Ed.). Studies in Logic and the Foundations of Mathematics, Vol. 137. Elsevier, 407–473.

[64] Anne S. Troelstra and Dirk van Dalen. 1988. *Constructivism in Mathematics: An Introduction*. Studies in Logic and the Foundations of

Mathematics, Vol. 121 and 123. North-Holland, Amsterdam. Two volumes; standard reference for BHK, Kripke semantics and realisability over an intuitionistic base.

[65] Nachiappan Valliappan. 2026. Lax Modal Lambda Calculi. In *34th EACSL Annual Conference on Computer Science Logic (CSL 2026) (LIPIcs)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. doi:10.4230/LIPIcs.CSL.2026.46 arXiv:2512.10779 Formal development: proof-relevant possible-world semantics and normalisation for lax calculi, machine-checked in Agda; the closest existing realiser-carrying Kripke model of lax logic.

[66] Nachiappan Valliappan, Fabian Ruch, and Carlos Tomé Cortiñas. 2022. Normalization for Fitch-Style Modal Calculi. *Proceedings of the ACM on Programming Languages* 6, ICFP (2022), 772–798. doi:10.1145/3547649 arXiv:2207.12807 Formal development: normalisation by evaluation for modal lambda calculi mechanised in Agda (`nachivpn/k`).

[67] Johan van Benthem and Eric Pacuit. 2011. Dynamic Logics of Evidence-Based Beliefs. *Studia Logica* 99, 1–3 (2011), 61–92.

[68] Iris van der Giessen and Ian Shillito. 2026. Uniform Interpolation with Constructive Diamond. arXiv:2602.16880 Formal development: uniform interpolation for CK and WK mechanised in Rocq; the Pitts-style mechanised-UI programme extending through the intuitionistic modal family, on whose natural path PLL lies.

[69] Jaap van Oosten. 2008. *Realizability: An Introduction to its Categorical Side*. Studies in Logic and the Foundations of Mathematics, Vol. 152. Elsevier, Amsterdam.

[70] Max Zorn. 1935. A Remark on Method in Transfinite Algebra. *Bull. Amer. Math. Soc.* 41, 10 (1935), 667–670. Cited as the maximality principle used by the classical F&M canonical model, and traded for the decision procedure in the finitised construction of the present paper.